

```

<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="utf-8">
<meta name="viewport" content="width=device-width, initial-scale=1">
<title>CyclePlanner</title>
<link rel="preconnect" href="https://fonts.googleapis.com">
<link rel="preconnect" href="https://fonts.gstatic.com" crossorigin>
<link
href="https://fonts.googleapis.com/css2?family=Quicksand:wght@400;500;600;700;800&di
splay=swap" rel="stylesheet">
<link href="assets/planning.css" rel="stylesheet">

<!-- SylCycle cloud: Firebase + shared auth -->
<script
src="https://www.gstatic.com/firebasejs/10.14.1/firebase-app-compat.js"></script>
<script
src="https://www.gstatic.com/firebasejs/10.14.1/firebase-auth-compat.js"></script>
<script
src="https://www.gstatic.com/firebasejs/10.14.1/firebase-firestore-compat.js"></scr
ipt>
<script src="sylcycle-auth.js"></script>

</head>
<body>
<div class="app">
  <header class="topbar">
    <div class="brand">
      <div class="brand-badge"></div>
      <div>
        <h1>Cycle<em>Planner</em></h1>
        <p>Focused planning workspace for Cycle 3 lessons, units and scope
maps.</p>
      </div>
    </div>
    <div class="top-actions">
      <span class="chip" id="typeChip"><svg viewBox="0 0 24 24" aria-hidden="true"
fill="none" stroke="currentColor" stroke-width="2" stroke-linecap="round"
stroke-linejoin="round"><path d="M4 19.5A2.5 2.5 0 0 1 6.5 17H20"/><path d="M6.5
2H20v20H6.5A2.5 2.5 0 0 1 4 19.5v-15A2.5 2.5 0 0 1 6.5 2z"/></svg><span>Lesson
plan</span></span>
      <a class="small-btn" href="index.html"><span>Cycle Suite</span></a>
      <button class="small-btn" data-go="library"><svg viewBox="0 0 24 24"
aria-hidden="true" fill="none" stroke="currentColor" stroke-width="2"
stroke-linecap="round" stroke-linejoin="round"><path d="M4 19.5A2.5 2.5 0 0 1 6.5
17H20"/><path d="M6.5 2H20v20H6.5A2.5 2.5 0 0 1 4 19.5v-15A2.5 2.5 0 0 1 6.5
2z"/></svg><span>Library</span></button>
      <button class="small-btn" data-go="settings"><svg viewBox="0 0 24 24"

```

```

aria-hidden="true" fill="none" stroke="currentColor" stroke-width="2"
stroke-linecap="round" stroke-linejoin="round"><path d="M12 15.5A3.5 3.5 0 1 0 12
8a3.5 3.5 0 0 0 7.5z"/><path d="M19.4 15a1.7 1.7 0 0 0 .34 1.87l.06.06a2 2 0 1
1-2.83 2.83l-.06-.06A1.7 1.7 0 0 0 15 19.4a1.7 1.7 0 0 0-1 .6 1.7 1.7 0 0 0-.4
1.1V21a2 2 0 1 1-4 0v-.09A1.7 1.7 0 0 0 8.6 19.4a1.7 1.7 0 0 0-1.87.34l-.06.06a2 2
0 1 1-2.83-2.83l.06-.06A1.7 1.7 0 0 0 4.6 15a1.7 1.7 0 0 0-.6-1 1.7 1.7 0 0
0-1.1-.4H3a2 2 0 1 1 0-4h.09A1.7 1.7 0 0 0 4.6 8.6a1.7 1.7 0 0
0-.34-1.87l-.06-.06a2 2 0 1 1 2.83-2.83l.06.06A1.7 1.7 0 0 0 9 4.6a1.7 1.7 0 0 0
1-.6 1.7 1.7 0 0 0 .4-1.1V3a2 2 0 1 1 4 0v.09A1.7 1.7 0 0 0 15.4 4.6a1.7 1.7 0 0 0
1.87-.34l.06-.06a2 2 0 1 1 2.83 2.83l-.06.06A1.7 1.7 0 0 0 19.4 9c.27.4.7.68
1.2.78.18.04.37.06.56.06H21a2 2 0 1 1 0 4h-.09A1.7 1.7 0 0 0 19.4
15z"/></svg><span>Settings</span></button>
</div>
</header>

<nav class="workflow-tabs" aria-label="CyclePlanner workflow">
  <button type="button" class="workflow-tab active" data-step="1"
data-go="setup"><strong>Planning Mode</strong><span>Choose shape, setup &
curriculum</span></button>
  <button type="button" class="workflow-tab" data-step="2"
data-go="create"><strong>Create</strong><span>Start blank, scaffold, template or
AI</span></button>
  <button type="button" class="workflow-tab" data-step="3"
data-go="edit"><strong>Edit & Review</strong><span>Edit parts, preview,
export</span></button>
</nav>

<section class="screen active" id="screen-setup">
  <div class="panel main-panel">
    <div class="stage-intro">
      <div>
        <div class="stage-kicker">Step 1</div>
        <h2>Planning Mode</h2>
        <p>Choose the planning shape, add the practical setup, then lock the
curriculum behaviour. Nothing else lives here.</p>
      </div>
      <button type="button" class="small-btn primary" data-go="create">Next:
Create</button>
    </div>

    <div class="section planning-type">
      <h3>Choose your planning</h3>
      <p class="sub">Select one planning shape. The rest of the workflow follows
this choice.</p>
      <div class="field" style="margin-top:14px">
        <label for="planningTypeSelect">Planning type</label>
        <select id="planningTypeSelect">
          <option value="lesson">Lesson plan – single lesson</option>
          <option value="unit">Unit plan – sequence of lessons</option>
          <option value="sequence">Scope & sequence – broad map</option>

```

```

        </select>
    </div>
    <p class="sub plan-type-description" id="planTypeDescription">A detailed
single lesson plan with flow, evidence, support, extension and light resources.</p>
</div>

<details class="section setup-details" id="setupDetails" open>
    <summary>
        <div>
            <h3 id="setupTitle">Lesson setup</h3>
            <p class="sub">The practical brief the planner needs before anything is
created.</p>
        </div>
        <span class="summary-chevron" aria-hidden="true">v</span>
    </summary>
    <div class="setup-body">
        <div class="grid-2 setup-grid" style="margin-top:14px">
            <div class="field">
                <label for="yearLevel">Year level</label>
                <select id="yearLevel"></select>
            </div>
            <div class="field setup-option" id="lessonLengthOption">
                <label for="durationPreset">Lesson length</label>
                <select id="durationPreset">
                    <option value="ai">AI suggested</option>
                    <option value="15 min">15 minutes</option>
                    <option value="30 min">30 minutes</option>
                    <option value="45 min">45 minutes</option>
                    <option value="60 min">60 minutes</option>
                    <option value="custom">Custom...</option>
                </select>
                <input id="durationCustom" class="duration-custom hidden"
placeholder="Custom length, e.g. 75 min, 90 min">
                <input id="durationInput" type="hidden" value="60 min">
            </div>
            <div class="field setup-option hidden" id="unitLessonOption">
                <label for="unitLessonPreset">Number of lessons</label>
                <select id="unitLessonPreset">
                    <option value="ai">AI suggested</option>
                    <option value="3">3 lessons</option>
                    <option value="4">4 lessons</option>
                    <option value="5">5 lessons</option>
                    <option value="6">6 lessons</option>
                    <option value="8">8 lessons</option>
                    <option value="10">10 lessons</option>
                    <option value="custom">Custom...</option>
                </select>
                <input id="unitLessonCustom" class="count-custom hidden"
placeholder="Custom number, e.g. 7, 12">
                <input id="unitLessonCount" type="hidden" value="6">
            </div>
        </div>
    </div>
</details>

```

```

    </div>
    <div class="field setup-option hidden" id="sequenceWeekOption">
      <label for="sequenceWeekPreset">Number of weeks</label>
      <select id="sequenceWeekPreset">
        <option value="ai">AI suggested</option>
        <option value="3">3 weeks</option>
        <option value="4">4 weeks</option>
        <option value="5">5 weeks</option>
        <option value="6">6 weeks</option>
        <option value="8">8 weeks</option>
        <option value="10">10 weeks</option>
        <option value="12">12 weeks</option>
        <option value="custom">Custom...</option>
      </select>
      <input id="sequenceWeekCustom" class="count-custom hidden"
placeholder="Custom number, e.g. 7, 14">
      <input id="sequenceWeekCount" type="hidden" value="6">
    </div>
  </div>
  <div class="field">
    <label for="briefInput">Prompt / planning brief</label>
    <textarea id="briefInput" placeholder="e.g. A CM1 English Literacy
lesson on using evidence to justify inferences, linked to Sherlock
Sam."></textarea>
  </div>
  <div class="field">
    <label for="extraContextInput">Extra context</label>
    <textarea id="extraContextInput" placeholder="Class needs, previous
lesson, timing constraints, text already used..."></textarea>
  </div>
</div>
</details>

  <div class="section">
    <h3>Curriculum</h3>
    <p class="sub" id="curriculumCycleNote">Same Cycle 3 spine as CycleStudio
and CycleAssess. No invented LO nonsense.</p>
    <div class="field">
      <label>Curriculum mode</label>
      <div class="segmented" id="curriculumModeButtons">
        <button type="button" class="seg-btn" data-curr="ai_both">AI chooses
all</button>
        <button type="button" class="seg-btn"
data-curr="manual_domain_ai_lo">Domain + AI LOs</button>
        <button type="button" class="seg-btn" data-curr="manual_both">Teacher
chooses</button>
      </div>
    </div>
    <div id="domainArea" class="field hidden">
      <label for="domainSelect"><span id="domainCycleLabel">Curriculum</span>

```

```

domain</label>
    <select id="domainSelect"></select>
</div>
<div id="outcomeArea" class="field hidden">
    <label>Learning outcomes</label>
    <div class="outcomes-box" id="outcomesBox"></div>
</div>
</div>

<details class="section setup-details" id="sourceDetails" open>
    <summary>
        <div>
            <h3>Source prompt material</h3>
            <p class="sub">Optional PDF, PPTX or image references for the AI to use
when creating the plan.</p>
        </div>
        <span class="summary-chevron" aria-hidden="true">v</span>
    </summary>
    <div class="setup-body">
        <input id="sourcePdfInput" class="hidden" type="file"
accept="application/pdf,.pdf">
        <input id="sourcePptxInput" class="hidden" type="file"
accept=".pptx,application/vnd.openxmlformats-officedocument.presentationml.presenta
tion">
        <input id="sourceImageInput" class="hidden" type="file" accept="image/*"
multiple>
        <div class="source-tools">
            <button type="button" class="small-btn"
data-action="choose-source-pdf">Upload PDF</button>
            <button type="button" class="small-btn"
data-action="choose-source-pptx">Upload PPTX</button>
            <button type="button" class="small-btn"
data-action="choose-source-image">Upload image</button>
            <button type="button" class="small-btn"
data-action="paste-source-image">Paste image</button>
        </div>
        <div class="source-drop" id="sourcePasteZone" tabindex="0">
            <strong>Paste screenshot or image here</strong>
            Click this box and press Ctrl/Cmd + V, or use the Paste image button
above.
        </div>
        <div class="field" style="margin-top:12px">
            <label for="sourceDetailsInput">Source details / instructions</label>
            <textarea id="sourceDetailsInput" placeholder="Tell the AI how to use
the uploaded material: extract the structure, follow this topic, adapt page 2 only,
use the image as inspiration, etc."></textarea>
        </div>
        <div class="source-list" id="sourceFileList"></div>
        <p class="tiny" style="margin-top:8px">Files are used as prompt context
for AI generation. PPTX slide text is extracted in the browser. Files are not saved

```

permanently in local storage.</p>

</div>

</details>

</div>

</section>

<section class="screen" id="screen-create">

<div class="panel main-panel">

<div class="stage-intro">

<div>

<div class="stage-kicker">Step 2</div>

<h2>Create</h2>

<p>Pick how this plan begins. After any creation action, you move to Edit & Review.</p>

</div>

<button type="button" class="small-btn" data-go="setup">Back to setup</button>

</div>

<div class="create-summary" id="createSummary"></div>

<div class="section">

<h3>Create</h3>

<p class="sub">Work from scratch, a scaffold, a saved template, or let AI draft the bones.</p>

<div class="grid-3" style="margin-bottom:10px">

<button class="btn secondary" data-action="start-blank">Start blank</button>

<button class="btn secondary" data-action="build-scaffold">Use planning scaffold</button>

<button class="btn outline-crimson" data-action="open-templates">Start from template</button>

</div>

<button class="btn crimson" id="generateBtn" data-action="generate">Generate with AI</button>

<div class="create-loading" id="createLoading" aria-live="polite">

<div class="spinner"></div>

<div>

<strong id="createLoadingTitle">Generating...

<p id="createLoadingText">This can take a little while for units and scope maps.</p>

</div>

</div>

<div class="status" id="statusBox"></div>

</div>

</div>

</section>

<section class="screen" id="screen-edit">

<div class="panel main-panel">

```

<div class="workspace-top">
  <div>
    <h2 id="workspaceTitle">Lesson workspace</h2>
    <p>Edit individual parts. Sections now collapse so the editor does not
turn into a scroll swamp. Preview updates live; export uses the preview.</p>
  </div>
  <div class="pill-row">
    <button class="small-btn" data-go="create">Back to create</button>
    <button class="small-btn" data-action="save-history">Save</button>
    <button class="small-btn" data-action="save-template">Save
template</button>
    <button class="small-btn" data-action="copy-text">Copy</button>
    <button class="small-btn export" data-action="export-pdf">Export
PDF</button>
    <button class="small-btn" data-action="print">Print</button>
  </div>
</div>

<div class="section edit-tools">
  <h3>Refine</h3>
  <p class="sub">Whole-plan actions live here. Section-specific tools now sit
inside the relevant editor sections.</p>
  <div class="field">
    <label for="modifyRequest">Modify with AI</label>
    <textarea id="modifyRequest" placeholder="Tell the AI exactly what to
change. Example: Make the hook shorter, add more EAL support, replace the exit
ticket, or make the assessment evidence more specific."></textarea>
  </div>
  <div class="grid-3" style="margin-bottom:10px">
    <button class="btn crimson" data-action="modify-ai">Modify with
AI</button>
    <button class="btn secondary" data-action="make-formal">Make
formal</button>
    <button class="btn secondary" data-action="make-quick">Make
quick</button>
  </div>
  <div class="grid-3">
    <button class="btn secondary" data-action="copy-studio-brief">Copy
CycleStudio brief</button>
    <button class="btn secondary" data-action="clear-modify">Clear AI
request</button>
    <button class="btn secondary" data-go="setup">Adjust setup</button>
  </div>
  <div class="status" id="editStatusBox"></div>
</div>

<div class="editor-preview" id="editorPreview">
  <div class="ai-overlay">
    <div class="ai-box">
      <div class="spinner"></div>

```

```

        <div><strong id="busyTitle">Working...</strong><div class="tiny"
id="busyText">The AI is drafting the plan.</div></div>
    </div>
</div>
<div class="editor-side">
    <div class="editor-local-tools">
        <strong>Editor sections</strong>
        <div class="pill-row">
            <button class="small-btn" data-action="expand-editor">Expand
all</button>
            <button class="small-btn" data-action="collapse-editor">Collapse
all</button>
        </div>
    </div>
    <div class="editor" id="editor"></div>
</div>
<div class="preview-wrap"><div id="preview"></div></div>
</div>
</div>
</section>

<section class="screen" id="screen-library">
    <div class="panel main-panel">
        <div class="workspace-top">
            <div><h2>Library</h2><p>Saved plans and templates. Local storage
only.</p></div>
            <div class="pill-row"><button class="small-btn" data-go="setup">Back to
planner</button><button class="small-btn danger" data-action="clear-library">Clear
all</button></div>
        </div>
        <div class="library-grid">
            <div class="section"><h3>History</h3><p class="sub">Recent saved/generated
plans.</p><div id="historyList"></div></div>
            <div class="section"><h3>Templates</h3><p class="sub">Reusable plan
structures.</p><div id="templateList"></div></div>
        </div>
    </div>
</section>

<section class="screen" id="screen-settings">
    <div class="panel main-panel">
        <div class="workspace-top">
            <div><h2>Settings</h2><p>Stored locally on this device.</p></div>
            <div class="pill-row"><button class="small-btn" data-go="setup">Back to
planner</button><button class="small-btn primary" data-action="save-settings">Save
settings</button></div>
        </div>
        <div class="section">
            <div class="grid-2">
                <div class="field"><label for="apiKey">Anthropic API key</label><input

```



```

id="apiKey" type="password" placeholder="sk-ant-api03-..."></div>
  <div class="field"><label for="modelName">Model ID</label><input
id="modelName" placeholder="claude-opus-4-5"></div>
    </div>
    <div class="field"><label for="preferredCycle">Preferred teaching
cycle</label><select id="preferredCycle"></select></div>
    <div class="field"><label for="defaultContext">Default planning
context</label><textarea id="defaultContext" placeholder="International French
School. English taught within French curriculum context. Prefer concise, practical
teacher notes."></textarea></div>
    <p class="tiny">The default context is sent with AI planning requests so
you do not keep retyping the same setup.</p>
    <p class="tiny">Your preferred cycle is remembered. Year level is still
chosen each time you use Planner.</p>
    <p class="tiny"><strong>Build:</strong> refactor/optimize checkpoint. Keep
doing this before adding large features.</p>
  </div>
  <div class="section maintenance-note">
    <h3>Refactor checkpoint</h3>
    <p class="sub">When the app gains a major feature, pause before the next
one and clean the structure. Tiny refactors now; no spaghetti archaeology
later.</p>
  </div>
</div>
</section>
</div>

<div class="modal" id="templateModal">
  <div class="modal-card">
    <div class="modal-head">
      <div><h3>Start from template</h3><p class="sub">Pick a saved template for
this planning type.</p></div>
      <button class="small-btn" data-action="close-templates">Close</button>
    </div>
    <div class="modal-body" id="templateModalList"></div>
  </div>
</div>

<div class="modal" id="cyclePreferenceModal">
  <div class="modal-card">
    <div class="modal-head">
      <div><h3>Choose your teaching cycle</h3><p class="sub">Planner will remember
this, and you can change it later in Settings.</p></div>
    </div>
    <div class="modal-body">
      <div class="cycle-choice-grid">
        <button class="btn secondary" data-cycle-choice="cycle1">Cycle 1</button>
        <button class="btn secondary" data-cycle-choice="cycle2">Cycle 2</button>
        <button class="btn crimson" data-cycle-choice="cycle3">Cycle 3</button>
      </div>
    </div>
  </div>
</div>

```

```
    <p class="tiny">Planner will show the matching year levels and curriculum  
bank for the cycle you choose.</p>
```

```
  </div>
```

```
</div>
```

```
</div>
```

```
<script
```

```
src="https://cdnjs.cloudflare.com/ajax/libs/jspdf/2.5.1/jspdf.umd.min.js"></script>
```

```
<script
```

```
src="https://cdnjs.cloudflare.com/ajax/libs/jszip/3.10.1/jszip.min.js"></script>
```

```
<script src="assets/cycle3-domain-library.js"></script>
```

```
<script src="assets/cycle-domain-library.js"></script>
```

```
<script src="assets/storage.js"></script>
```

```
<script>
```

```
/* =====
```

```
  CyclePlanner JS – organised by responsibility (refactor/optimise pass 2)
```

```
  1. Data + state
```

```
  2. Utilities + storage
```

```
  3. Curriculum rendering
```

```
  4. Plan factory + normalisation
```

```
  5. Navigation + setup
```

```
  6. Editor rendering
```

```
  7. Preview rendering
```

```
  8. AI actions
```

```
  9. Library + settings
```

```
  10. Event binding
```

```
===== */
```

```
/* 1. Data + state */
```

```
const CYCLE_LABELS = {
```

```
  cycle1:'Cycle 1',
```

```
  cycle2:'Cycle 2',
```

```
  cycle3:'Cycle 3'
```

```
};
```

```
const CYCLE_YEAR_LEVELS = {
```

```
  cycle1:['PS', 'MS', 'GS'],
```

```
  cycle2:['CP', 'CE1', 'CE2'],
```

```
  cycle3:['CM1', 'CM2', 'CM1/CM2']
```

```
};
```

```
const CYCLE_DOMAIN_LIBRARIES = window.CYCLE_DOMAIN_LIBRARIES ||
```

```
{cycle3>window.CYCLE3_DOMAIN_LIBRARY || {}};
```

```
let currentCycle = 'cycle3';
```

```
let DOMAIN_LIBRARY = CYCLE_DOMAIN_LIBRARIES.cycle3 || {};
```

```
let DOMAIN_NAMES = Object.keys(DOMAIN_LIBRARY);
```

```
const APP_VERSION = '0.2.1-refactor-optimised';
```

```
const PLAN_LABELS = {
```

```
  lesson:'Lesson plan',
```

```
  unit:'Unit plan',
```

```
  sequence:'Scope & sequence'
```

```

};
const PLAN_DESCRIPTIONS = {
  lesson: 'A detailed single lesson plan with flow, evidence, support, extension and
light resources.',
  unit: 'A connected sequence of lessons with shared outcomes, lesson-by-lesson
focus and assessment evidence.',
  sequence: 'A broad map across weeks or blocks, useful for coverage, pacing and
scope planning.'
};
function planTypeChipHtml(type=state.type){
  return `

```

```

const qs = id => document.getElementById(id);
const qsa = (sel, root=document) => Array.from(root.querySelectorAll(sel));
function debounce(fn, delay=SAVE_DEBOUNCE_MS){
  let timer = 0;
  return (...args) => {
    clearTimeout(timer);
    timer = setTimeout(() => fn(...args), delay);
  };
}
function clonePlain(value){
  return value == null ? value : JSON.parse(JSON.stringify(value));
}
function clampInt(value, min=1, max=40, fallback=1){
  const parsed = parseInt(value, 10);
  if(!Number.isFinite(parsed)) return fallback;
  return Math.max(min, Math.min(max, parsed));
}
function cleanText(value='') {
  return String(value || '')
    .replace(/\r\n/g, '\n')
    .replace(/([,;:!?])(?=\S)/g, '$1 ')
    .replace(/\.([A-Z])/g, '. $1')
    .replace(/[ \t]+/g, ' ')
    .replace(/\n[ \t]+/g, '\n')
    .replace(/\n{3,}/g, '\n\n')
    .trim();
}
function esc(value='') {
  return
  cleanText(value).replaceAll('&', '&').replaceAll('<', '<').replaceAll('>', '>')
  .replaceAll('"', '"').replaceAll("'", ''');
}
function nl(value='') {
  return esc(value).replace(/\n/g, '<br>');
}
function splitCompactSteps(value='', limit=4){
  const text = cleanText(value)
  // Turn numbered list markers into line breaks before splitting.
  .replace(/(^|\n\s)\d+([.])\s+/g, '\n')
  // Remove orphan bullets such as "• 1" or a line containing only "2".
  .replace(/(^|\n)\s*(?:[-•]\s*)?\d+\s*(?=\n|$)/g, '\n')
  .trim();
  if(!text) return [];
  const parts = text
    .split(/(?:\n+|\s*\s*(?<=[.!?])\s+(?=[A-Z]))/g)
    .map(part => part.trim()
      .replace(/^[ -•]\s*/, '')
      .replace(/^\d+([.])?\s*/, '')
      .replace(/[.;]$/, '')
      .trim())

```

```

        .filter(part => part && !/^d+$/.test(part));
    if(parts.length <= 1) return parts.length ? parts : [text];
    return parts.slice(0, limit);
}
function compactCell(value='', limit=4){
    const text = cleanText(value);
    if(!text) return '';
    const parts = splitCompactSteps(text, limit);
    if(parts.length > 1){
        return `<ul
class="compact-list">${parts.map(part=>`<li>${esc(part)}</li>`).join('')}</ul>`;
    }
    return nl(text);
}
function listHtml(items=[]){
    return items.length ?
`<ul>${items.map(item=>`<li>${esc(item)}</li>`).join('')}</ul>` : '<p
class="tiny">None yet.</p>';
}
function readJSON(key, fallback){
    if(window.SylCycleStorage) return SylCycleStorage.readJson(key, fallback);
    try{return JSON.parse(localStorage.getItem(key) || JSON.stringify(fallback));}
    catch(err){return fallback;}
}
function writeJSON(key, value){
    if(window.SylCycleStorage) SylCycleStorage.writeJson(key, value);
    else {
        try{ localStorage.setItem(key, JSON.stringify(value)); }
        catch(err){ console.warn('CyclePlanner storage write failed:', err); }
    }
    if(window.SylCycle) SylCycle.cloudSave(key, value);
}
function normaliseCycle(value){
    return CYCLE_LABELS[value] ? value : 'cycle3';
}
function getCycleLabel(cycle=currentCycle){
    return CYCLE_LABELS[normaliseCycle(cycle)];
}
function getCycleYears(cycle=currentCycle){
    return CYCLE_YEAR_LEVELS[normaliseCycle(cycle)] || CYCLE_YEAR_LEVELS.cycle3;
}
function hasStoredCyclePreference(){
    const stored = readJSON(STORAGE.prefs, {});
    return Object.prototype.hasOwnProperty.call(stored, 'preferredCycle');
}
function setCurrentCycle(cycle){
    currentCycle = normaliseCycle(cycle);
    DOMAIN_LIBRARY = CYCLE_DOMAIN_LIBRARIES[currentCycle] ||
CYCLE_DOMAIN_LIBRARIES.cycle3 || {};
    DOMAIN_NAMES = Object.keys(DOMAIN_LIBRARY);
}

```

```

}
function populatePreferredCycleSelect(){
  const select = qs('preferredCycle');
  if(!select) return;
  select.innerHTML = Object.entries(CYCLE_LABELS)
    .map(([value,label])=><option value="${esc(value)}">${esc(label)}</option>`)
    .join('');
  select.value = currentCycle;
}
function populateYearLevels(preferredYear='') {
  const select = qs('yearLevel');
  if(!select) return;
  const years = getCycleYears();
  select.innerHTML = years.map(year=><option
value="${esc(year)}">${esc(year)}</option>`).join('');
  select.value = years.includes(preferredYear) ? preferredYear : years[0];
}
function applyCyclePreference(cycle, preferredYear='') {
  setCurrentCycle(cycle);
  populatePreferredCycleSelect();
  populateYearLevels(preferredYear);
  populateDomains();
  const domainSelect = qs('domainSelect');
  if(domainSelect && (!DOMAIN_LIBRARY[domainSelect.value] ||
!domainHasOutcomes(domainSelect.value))) {
    const fallbackDomain = availableDomainNames()[0] || DOMAIN_NAMES[0] || '';
    domainSelect.value = fallbackDomain;
  }
  state.selectedOutcomes = state.selectedOutcomes.filter(outcome =>
getFlat(domainSelect?.value || '').includes(outcome));
  if(qs('domainCycleLabel')) qs('domainCycleLabel').textContent = 'Curriculum';
  if(qs('curriculumCycleNote')) {
    qs('curriculumCycleNote').textContent = `${getCycleLabel()} selected. Choose a
domain, or let AI choose from the available curriculum bank.`;
  }
  renderCurriculumMode();
  renderCreateSummary();
}
function unique(items=[]){
  return [...new Set(items.filter(Boolean))];
}
function splitLines(value='') {
  return String(value).split('\n').map(x=>x.trim()).filter(Boolean);
}
function joinLines(items=[]){
  return (items || []).join('\n');
}
function makeId(){
  return window.crypto?.randomUUID ? window.crypto.randomUUID() :
`cp_${Date.now()}_${Math.random().toString(16).slice(2)}`;
}

```

```

}
function fileToDataURL(file){
  return new Promise((resolve,reject)=>{
    const reader = new FileReader();
    reader.onload = () => resolve(reader.result);
    reader.onerror = reject;
    reader.readAsDataURL(file);
  });
}
function sourceKind(file){
  const type = file?.type || '';
  if(type.includes('pdf') || /\.pdf$/i.test(file?.name || '')) return 'pdf';
  if(type.includes('presentationml') || /\.pptx$/i.test(file?.name || '')) return
'pptx';
  if(type.startsWith('image/')) return 'image';
  return 'file';
}
function decodeXmlEntities(str='') {
  const map = {amp:'&', lt:'<', gt:'>', quot:'"', apos:'"'};
  return String(str || '')
    .replace(/&#(\d+);/g, (_, n) => String.fromCharCode(parseInt(n, 10)))
    .replace(/&#x([0-9a-f]+)/gi, (_, n) => String.fromCharCode(parseInt(n, 16)))
    .replace(/&([a-z]+)/gi, (_, entity) => map[entity] || `&${entity};`);
}
async function extractPptxText(file){
  if(!window.JSZip) throw new Error('PPTX support did not load. Try again in a
moment, or export the slides as PDF.');
```

```

  const zip = await window.JSZip.loadAsync(await file.arrayBuffer());
  const slideFiles = Object.keys(zip.files)
    .filter(name => /^ppt\/slides\/slide\d+\.xml$/i.test(name))
    .sort((a,b) => (parseInt(a.match(/slide(\d+)\.xml/i)?.[1] || '0', 10) -
parseInt(b.match(/slide(\d+)\.xml/i)?.[1] || '0', 10)));
  const slides = [];
  for(const name of slideFiles){
    const xml = await zip.files[name].async('string');
    const bits = Array.from(xml.matchAll(/<a:t[^\>]*>([\s\S]*)<\a:t>/g))
      .map(match => decodeXmlEntities(match[1]).replace(/\s+/g, ' ').trim())
      .filter(Boolean);
    if(bits.length) slides.push(`Slide ${slides.length + 1}: ${bits.join(' ')}`);
  }
  const text = slides.join('\n').trim();
  if(!text) throw new Error('No readable text was found in that PPTX. If it is
mostly visual, export it as PDF instead.');
```

```

  return text;
}
async function addSourceFile(file){
  if(!file) return false;
  const kind = sourceKind(file);
  if(kind !== 'pdf' && kind !== 'image' && kind !== 'pptx') return false;
  const extractedText = kind === 'pptx' ? await extractPptxText(file) : '';

```

```

const dataUrl = await fileToDataUrl(file);
state.sourceFiles.push({
  id: makeId(),
  name: file.name || (kind === 'pdf' ? 'Uploaded PDF' : kind === 'pptx' ?
'Uploaded PPTX' : 'Uploaded image'),
  kind,
  mimeType: file.type || (kind === 'pdf' ? 'application/pdf' : kind === 'pptx' ?
'application/vnd.openxmlformats-officedocument.presentationml.presentation' :
'image/png'),
  dataUrl,
  base64: String(dataUrl).split(',')[1] || '',
  extractedText
});
renderSourceFiles();
return true;
}
async function addSourceFiles(files){
  let added = 0;
  for(const file of Array.from(files || [])){
    if(await addSourceFile(file)) added++;
  }
  return added;
}
function removeSourceFile(id){
  state.sourceFiles = state.sourceFiles.filter(file => file.id !== id);
  renderSourceFiles();
}
function sourcePromptText(){
  return state.sourceFiles
    .filter(file => file.kind === 'pptx' && file.extractedText)
    .map(file => `PPTX source: ${file.name}\n${file.extractedText}`)
    .join('\n\n')
    .slice(0, 24000);
}
function renderSourceFiles(){
  const root = qs('sourceFileList');
  if(!root) return;
  if(!state.sourceFiles.length){
    root.innerHTML = '<p class="tiny">No source files added yet.</p>';
    return;
  }
  root.innerHTML = state.sourceFiles.map(file=>`
    <div class="source-item">
      <div class="source-file-left">
        ${file.kind === 'image' ? `` : `<div class="source-icon">${file.kind === 'pptx' ?
'PPTX' : 'PDF'}</div>`}
        <div><strong>${esc(file.name)}</strong><span>${file.kind === 'pdf' ? 'PDF
source' : file.kind === 'pptx' ? 'PPTX slide text' : 'Image source'} · used for AI
generation</span></div>

```



```

        </div>
        <button type="button" class="mini-btn danger"
data-source-remove="${esc(file.id)}">Remove</button>
    </div>
    `).join('');
}
async function addSourceImagesFromClipboardItems(items){
    let added = 0;
    for(const item of Array.from(items || [])){
        if(item.kind === 'file' && String(item.type || '').startsWith('image/')){
            const file = item.getAsFile?();
            if(file && await addSourceFile(file)) added++;
        }
    }
    return added;
}
async function addSourceImagesFromClipboardApi(items){
    let added = 0;
    for(const item of Array.from(items || [])){
        const imageType = (item.types || []).find(type => type.startsWith('image/'));
        if(!imageType) continue;
        const blob = await item.getType(imageType);
        const file = new File([blob], `Pasted source image ${state.sourceFiles.length +
added + 1}.${imageType.split('/')[1] || 'png'}`, {type:imageType});
        if(await addSourceFile(file)) added++;
    }
    return added;
}
function setStatus(message, kind='ok', target='statusBox'){
    const box = qs(target);
    if(!box) return;
    box.textContent = message;
    box.className = `status show ${kind}`;
}
function clearStatus(){
    ['statusBox', 'editStatusBox'].forEach(id=>{
        const box = qs(id);
        if(box){ box.textContent=''; box.className='status'; }
    });
}
function setBusy(on, title='Working...', text='One reasonably obedient robot is
thinking.'){
    qs('editorPreview')?.classList.toggle('busy', !!on);
    if(qs('busyTitle')) qs('busyTitle').textContent = title;
    if(qs('busyText')) qs('busyText').textContent = text;
}

let previewRenderFrame = 0;
function persistCurrentPlan(){
    if(state.plan) writeJSON(STORAGE.current, clonePlain(state.plan));
}

```

```

}
const persistCurrentPlanSoon = debounce(persistCurrentPlan);
function schedulePreviewUpdate(){
  if(previewRenderFrame) cancelAnimationFrame(previewRenderFrame);
  previewRenderFrame = requestAnimationFrame(()=>{
    previewRenderFrame = 0;
    renderPreview();
    persistCurrentPlanSoon();
  });
}
function renderAllAndPersist(){
  renderAll();
  persistCurrentPlan();
}

function getGenerationLabel(){
  if(state.type === 'unit') return isCountAiSuggested('unitLesson') ? 'AI-suggested
unit plan' : `${getUnitLessonCount()}-lesson unit plan`;
  if(state.type === 'sequence') return isCountAiSuggested('sequenceWeek') ?
'AI-suggested scope & sequence' : `${getSequenceWeekCount()}-week scope &
sequence`;
  return isDurationAiSuggested() ? 'AI-suggested lesson plan' :
`${getDurationValue()} lesson plan`;
}
function setCreateBusy(on, title='Generating...', text='This can take a little
while.'){
  const box = qs('createLoading');
  const btn = qs('generateBtn');
  if(box) box.classList.toggle('show', !!on);
  if(qs('createLoadingTitle')) qs('createLoadingTitle').textContent = title;
  if(qs('createLoadingText')) qs('createLoadingText').textContent = text;
  if(btn){
    btn.disabled = !!on;
    btn.textContent = on ? 'Generating...' : 'Generate with AI';
  }
}
function getPrefs(){
  return {...DEFAULT_PREFS, ...readJSON(STORAGE.prefs, {})};
}
function collectPrefs(){
  return {
    ...getPrefs(),
    preferredCycle:currentCycle,
    yearLevel:qs('yearLevel').value,
    duration:getDurationValue(),
    unitLessonCount:getUnitLessonCount(),
    sequenceWeekCount:getSequenceWeekCount(),
    brief:qs('briefInput').value,
    extraContext:qs('extraContextInput').value,
    sourceDetails:qs('sourceDetailsInput')?.value || '',
  }
}

```

```

    planningType:qs('planningTypeSelect').value,
    curriculumMode:state.curriculumMode,
    domain:qs('domainSelect').value,
    apiKey:qs('apiKey').value,
    modelName:qs('modelName').value || 'claude-opus-4-5',
    defaultContext:qs('defaultContext').value
  };
}
function savePrefs(){
  writeJSON(STORAGE.prefs, collectPrefs());
}
const savePrefsSoon = debounce(savePrefs);
function loadPrefs(){
  const p = getPrefs();
  applyCyclePreference(p.preferredCycle, p.yearLevel);
  setDurationUI(p.duration);
  setCountUI('unitLesson', p.unitLessonCount || '6');
  setCountUI('sequenceWeek', p.sequenceWeekCount || '6');
  qs('briefInput').value = p.brief;
  qs('extraContextInput').value = p.extraContext;
  if(qs('sourceDetailsInput')) qs('sourceDetailsInput').value = p.sourceDetails ||
  '';
  qs('planningTypeSelect').value = p.planningType;
  qs('apiKey').value = p.apiKey;
  qs('modelName').value = p.modelName;
  qs('defaultContext').value = p.defaultContext;
  state.type = p.planningType;
  state.curriculumMode = p.curriculumMode;
}
const AI_SUGGESTED_VALUE = 'ai';
const AI_SUGGESTED_LABEL = 'AI suggested';
const STANDARD_DURATIONS = ['15 min', '30 min', '45 min', '60 min'];
function isAiSuggestedValue(value){
  return String(value || '').trim().toLowerCase() === AI_SUGGESTED_VALUE;
}
function concreteSetupValue(value){
  return isAiSuggestedValue(value) ? '' : value;
}
function isDurationAiSuggested(){ return qs('durationPreset')?.value ===
AI_SUGGESTED_VALUE; }
function isCountAiSuggested(kind){ return qs(`${kind}Preset`)?.value ===
AI_SUGGESTED_VALUE; }
function getDurationValue(){
  const preset = qs('durationPreset')?.value || '';
  if(preset === AI_SUGGESTED_VALUE) return AI_SUGGESTED_VALUE;
  if(preset === 'custom') return (qs('durationCustom')?.value || '').trim() ||
'Custom';
  if(preset) return preset;
  return qs('durationInput')?.value || '60 min';
}

```

```

function setDurationUI(value='60 min'){
  const clean = String(value || '60 min').trim();
  const preset = qs('durationPreset');
  const custom = qs('durationCustom');
  const hidden = qs('durationInput');
  if(!preset || !custom || !hidden) return;
  if(isAiSuggestedValue(clean)){
    preset.value = AI_SUGGESTED_VALUE;
    custom.value = '';
    custom.classList.add('hidden');
    hidden.value = AI_SUGGESTED_VALUE;
  }else if(STANDARD_DURATIONS.includes(clean)){
    preset.value = clean;
    custom.value = '';
    custom.classList.add('hidden');
    hidden.value = clean;
  }else{
    preset.value = 'custom';
    custom.value = clean;
    custom.classList.remove('hidden');
    hidden.value = clean;
  }
}

function syncDurationUI(){
  const preset = qs('durationPreset');
  const custom = qs('durationCustom');
  const hidden = qs('durationInput');
  if(!preset || !custom || !hidden) return;
  custom.classList.toggle('hidden', preset.value !== 'custom');
  hidden.value = getDurationValue();
}

function handleDurationChange(){
  syncDurationUI();
  savePrefs();
  renderCreateSummary();
  if(state.plan){
    if(!isDurationAiSuggested()) state.plan.duration = getDurationValue();
    schedulePreviewUpdate();
  }
}

const STANDARD_UNIT_LESSON_COUNTS = ['3','4','5','6','8','10'];
const STANDARD_SEQUENCE_WEEK_COUNTS = ['3','4','5','6','8','10','12'];
function getCountValue(kind){
  const preset = qs(`${kind}Preset`)?.value || '';
  const custom = qs(`${kind}Custom`)?.value || '';
  const hidden = qs(`${kind}Count`);
  if(preset === AI_SUGGESTED_VALUE) return AI_SUGGESTED_VALUE;
  if(preset === 'custom') return String(custom || hidden?.value || '').trim() ||
'6';

```

```

    if(preset) return preset;
    return hidden?.value || '6';
}
function getUnitLessonCount(){ return getCountValue('unitLesson'); }
function getSequenceWeekCount(){ return getCountValue('sequenceWeek'); }
function setCountUI(kind, value='6'){
    const clean = String(value || '6').trim();
    const preset = qs(`${kind}Preset`);
    const custom = qs(`${kind}Custom`);
    const hidden = qs(`${kind}Count`);
    if(!preset || !custom || !hidden) return;
    const standard = kind === 'sequenceWeek' ? STANDARD_SEQUENCE_WEEK_COUNTS :
STANDARD_UNIT_LESSON_COUNTS;
    if(isAiSuggestedValue(clean)){
        preset.value = AI_SUGGESTED_VALUE;
        custom.value = '';
        custom.classList.add('hidden');
        hidden.value = AI_SUGGESTED_VALUE;
    }else if(standard.includes(clean)){
        preset.value = clean;
        custom.value = '';
        custom.classList.add('hidden');
        hidden.value = clean;
    }else{
        preset.value = 'custom';
        custom.value = clean;
        custom.classList.remove('hidden');
        hidden.value = clean;
    }
}
function syncCountUI(kind){
    const preset = qs(`${kind}Preset`);
    const custom = qs(`${kind}Custom`);
    const hidden = qs(`${kind}Count`);
    if(!preset || !custom || !hidden) return;
    custom.classList.toggle('hidden', preset.value !== 'custom');
    hidden.value = getCountValue(kind);
}
function handleCountChange(kind){
    syncCountUI(kind);
    savePrefs();
    renderCreateSummary();
    if(state.plan){
        if(kind === 'unitLesson' && !isCountAiSuggested('unitLesson'))
state.plan.lessonCount = getUnitLessonCount();
        if(kind === 'sequenceWeek' && !isCountAiSuggested('sequenceWeek'))
state.plan.weekCount = getSequenceWeekCount();
        schedulePreviewUpdate();
    }
}

```

```

function getPlanningScaleText(type=state.type){
  if(type === 'unit') return isCountAiSuggested('unitLesson') ? AI_SUGGESTED_LABEL
: `${getUnitLessonCount()} lessons`;
  if(type === 'sequence') return isCountAiSuggested('sequenceWeek') ?
AI_SUGGESTED_LABEL : `${getSequenceWeekCount()} weeks`;
  return isDurationAiSuggested() ? AI_SUGGESTED_LABEL : getDurationValue();
}
function syncSetupControls(type=state.type){
  qs('lessonLengthOption')?.classList.toggle('hidden', type !== 'lesson');
  qs('unitLessonOption')?.classList.toggle('hidden', type !== 'unit');
  qs('sequenceWeekOption')?.classList.toggle('hidden', type !== 'sequence');
}
function setNested(root, path, value){
  const parts = path.split('.');
  let obj = root;
  for(let i=0;i<parts.length-1;i++){
    const key = parts[i];
    if(obj[key] === undefined || obj[key] === null) obj[key] =
/^\\d+$/.test(parts[i+1]) ? [] : {};
    obj = obj[key];
  }
  obj[parts[parts.length - 1]] = value;
}

/* 3. Curriculum rendering */
function getGrouped(domain){
  const entry = DOMAIN_LIBRARY[domain];
  return entry && !Array.isArray(entry) && typeof entry === 'object' ? entry :
null;
}
function getFlat(domain){
  const entry = DOMAIN_LIBRARY[domain];
  if(Array.isArray(entry)) return entry;
  if(entry && typeof entry === 'object') return Object.values(entry).flat();
  return [];
}
function domainHasOutcomes(domain){
  return getFlat(domain).length > 0;
}
function availableDomainNames(){
  return DOMAIN_NAMES.filter(domainHasOutcomes);
}
function selectedDomainHasOutcomes(){
  return domainHasOutcomes(qs('domainSelect')?.value || '');
}
const MATCH_STOPWORDS = new
Set(['and','the','for','with','from','into','their','them','this','that','these','t
hose','using','use','your','about','across','within','between','through','around','
while','where','when','what','which','over','under','such','than','then','they','ha
ve','has','had','its','our','out','off','all','too','can','may','how','will','are',

```

```

'was','were']]);
function normaliseForMatch(value='') {
  return String(value || '')
    .normalize('NFD').replace(/[\u0300-\u036f]/g, '')
    .toLowerCase().replace(/&/g, ' and ')
    .replace(/[^a-z0-9]+/g, ' ')
    .trim();
}
function meaningfulTokens(value='') {
  return unique(normaliseForMatch(value).split(/\s+/).filter(t => t && t.length > 2
&& !MATCH_STOPWORDS.has(t)));
}
function scoreTextAgainstTokens(textNorm, tokens) {
  let score = 0;
  (tokens || []).forEach(token => { if(textNorm.includes(token)) score +=
token.length >= 7 ? 2.3 : 1.25; });
  return score;
}
function canonicalDomain(value='') {
  const raw = String(value || '').trim();
  if(!raw) return '';
  if(currentCycle === 'cycle2' && raw === 'French') return 'Literacy';
  if(DOMAIN_NAMES.includes(raw)) return raw;
  const norm = normaliseForMatch(raw);
  return DOMAIN_NAMES.find(name => normaliseForMatch(name) === norm) ||
DOMAIN_NAMES.find(name => normaliseForMatch(name).includes(norm) ||
norm.includes(normaliseForMatch(name))) || '';
}
function domainKeywordSeed(domain) {
  switch(domain) {
    case 'Science and Technology': return
['science','scientific','light','shadow','shadows','source','sources','torch','opaq
ue','transparent','translucent','energy','sound','electricity','force','motion','ma
terial','materials','experiment','investigation','observe','observations','model'];
    case 'English Literacy': return
['reading','read','writing','write','sentence','grammar','spelling','punctuation','
vocabulary','inference','comprehension','text','author'];
    case 'Literacy': return
['reading','read','writing','write','sentence','grammar','spelling','punctuation','
vocabulary','inference','comprehension','text','oral','speak','speaking'];
    case 'Mathematics': return
['number','calculate','calculation','fraction','decimal','measure','measurement','g
eometry','shape','angle','area','perimeter','graph','data'];
    case 'History and Geography': return
['history','geography','map','place','settlement','land','resource','resources','ti
meline','travel','region','country'];
    case 'Civics and Moral Education': return
['citizen','rights','responsibilities','rules','debate','republic','discrimination'
,'ethical'];
    case 'Visual Arts': return

```

```

['art','artist','artwork','colour','color','shape','texture','painting','drawing'];
  case 'Music': return
['music','rhythm','tempo','pitch','sound','sing','compose'];
  case 'PE': return ['sport','physical','movement','team','game','fitness'];
  case 'Modern Languages': return
['language','speaking','listening','cultural','vocabulary'];
  default: return [];
}
}
function inferDomainFromPlan(plan={}){
  const textNorm = normaliseForMatch(JSON.stringify(plan || {}));
  if(!textNorm) return '';
  let best = '';
  let bestScore = -Infinity;
  DOMAIN_NAMES.forEach(domain => {
    let score = scoreTextAgainstTokens(textNorm, meaningfulTokens(domain)) * 1.3;
    score += scoreTextAgainstTokens(textNorm, domainKeywordSeed(domain)) * 1.8;
    getFlat(domain).forEach(outcome => { score += Math.min(7,
scoreTextAgainstTokens(textNorm, meaningfulTokens(outcome)) * .38); });
    if(score > bestScore){ bestScore = score; best = domain; }
  });
  return bestScore > 1 ? best : '';
}
function chooseOutcomesForPlan(domain, plan={}, maxCount=3){
  const allowed = getFlat(domain);
  if(!allowed.length) return [];
  const textNorm = normaliseForMatch(JSON.stringify(plan || {}));
  const ranked = allowed.map(outcome => {
    let score = scoreTextAgainstTokens(textNorm, meaningfulTokens(outcome));
    if(domain === 'Science and Technology' &&
/b(light|shadow|shadows|torch|source|sources)/b.test(textNorm)){
      if(/observations? and simple models|everyday physical
phenomena/i.test(outcome)) score += 9;
      if(/scientific vocabulary/i.test(outcome)) score += 7;
      if(/Ask questions/i.test(outcome)) score += 5;
      if(/Plan and carry out/i.test(outcome)) score += 4;
      if(/light, sound or electrical signals/i.test(outcome)) score += 3;
    }
    return {outcome, score};
  }).sort((a,b)=>b.score-a.score);
  return ranked.filter(x=>x.score > 0).slice(0, maxCount).map(x=>x.outcome);
}
function repairCurriculum(plan){
  if(!plan) return plan;
  const inferred = inferDomainFromPlan(plan);
  const current = canonicalDomain(plan.domain);
  const hasUsefulOutcomes = Array.isArray(plan.learningOutcomes) &&
plan.learningOutcomes.length > 0;
  if(state.curriculumMode === 'ai_both'){
    if(inferred && (!current || !hasUsefulOutcomes || (current === 'English

```



```

Literacy' && inferred !== 'English Literacy'))){
  plan.domain = inferred;
}else{
  plan.domain = current || inferred || '';
}
if(!plan.learningOutcomes?.length && plan.domain){
  plan.learningOutcomes = chooseOutcomesForPlan(plan.domain, plan, plan.type
=== 'unit' ? 4 : 3);
}
}else{
  plan.domain = current || qs('domainSelect')?.value || '';
}
return plan;
}
function populateDomains(){
  qs('domainSelect').innerHTML = DOMAIN_NAMES.map(name=>{
    const pending = !domainHasOutcomes(name);
    return `<option value="${esc(name)}" ${pending ? 'disabled' :
''}>${esc(name)}${pending ? ' (not available yet)' : ''}</option>`;
  }).join('');
}
function renderCurriculumMode(){
  qsa('[data-curr]').forEach(btn=>btn.classList.toggle('active', btn.dataset.curr
=== state.curriculumMode));
  qs('domainArea').classList.toggle('hidden', state.curriculumMode === 'ai_both');
  qs('outcomeArea').classList.toggle('hidden', state.curriculumMode !==
'manual_both');
  if(state.curriculumMode === 'manual_both') renderOutcomePicker();
}
function renderOutcomePicker(){
  const domain = qs('domainSelect').value;
  const selected = new Set(state.selectedOutcomes);
  const grouped = getGrouped(domain);
  if(!domainHasOutcomes(domain)){
    qs('outcomesBox').innerHTML = `<p class="tiny">This domain is not available
yet.</p>`;
    return;
  }
  const row = outcome => `
    <label class="outcome-option">
      <input type="checkbox" data-outcome="${esc(outcome)}" ${selected.has(outcome)
? 'checked' : ''}>
      <span>${esc(outcome)}</span>
    </label>`;
  qs('outcomesBox').innerHTML = grouped
  ? Object.entries(grouped).map(([strand,outcomes])=>`<div
class="outcome-group"><div
class="outcome-group-title">${esc(strand)}</div>${outcomes.map(row).join('')}</div>
`).join('')
  : getFlat(domain).map(row).join('');
}

```

```

}
function applyCurriculumToPlan(){
  if(!state.plan) return;
  if(state.curriculumMode === 'ai_both') return;
  state.plan.domain = qs('domainSelect').value;
  if(state.curriculumMode === 'manual_both') state.plan.learningOutcomes =
[...state.selectedOutcomes];
}
function catalogue(domain='') {
  const domains = domain && DOMAIN_LIBRARY[domain] ? [domain] :
availableDomainNames();
  return domains.map(name=>{
    const grouped = getGrouped(name);
    if(grouped){
      return `${name}:\n` + Object.entries(grouped).map(([strand,outcomes])=>`
${strand}:\n` + outcomes.map(o=>`    - "${o}"`).join('\n')).join('\n');
    }
    return `${name}:\n` + getFlat(name).map(o=>`    - "${o}"`).join('\n');
  }).join('\n\n');
}

/* 4. Plan factory + normalisation */
function baseFields(type){
  return {
    type,
    title:'Untitled ' + PLAN_LABELS[type].toLowerCase(),
    yearLevel:qs('yearLevel')?.value || 'CM1',
    duration:isDurationAiSuggested() ? '' : getDurationValue(),
    lessonCount: type === 'unit' ? (isCountAiSuggested('unitLesson') ? '' :
getUnitLessonCount()) : '',
    weekCount: type === 'sequence' ? (isCountAiSuggested('sequenceWeek') ? '' :
getSequenceWeekCount()) : '',
    domain: state.curriculumMode === 'ai_both' ? '' : qs('domainSelect')?.value ||
'',
    learningOutcomes: state.curriculumMode === 'manual_both' ?
[...state.selectedOutcomes] : [],
    context:qs('extraContextInput')?.value || ''
  };
}
function blankPlan(type=state.type){
  if(type === 'unit'){
    return {
      ...baseFields('unit'),
      lessonCount:isCountAiSuggested('unitLesson') ? '' : getUnitLessonCount(),
      overview:'',
      drivingQuestion:'',
      vocabulary:[],
      lessons:[],
      differentiation:{support:'',extension:'',eal:''},
      finalAssessment:'',

```

```

        cycleAssessHooks:{evidenceToCollect:''},
        notes:''
    };
}
if(type === 'sequence'){
    return {
        ...baseFields('sequence'),
        weekCount:isCountAiSuggested('sequenceWeek') ? '' : getSequenceWeekCount(),
        overview:'',
        termOrBlock:'',
        weeks:[],
        coverageNotes:'',
        cycleAssessHooks:{coverageNotes:''},
        notes:''
    };
}
return {
    ...baseFields('lesson'),
    subject:'',
    objectives:[],
    materials:[],
    vocabulary:[],
    priorKnowledge:'',
    lessonFlow:[],
    differentiation:{support:'',extension:'',eal:''},
    keyQuestions:[],
    exitTicket:'',
    misconceptionCheck:'',
    assessment:{evidence:'',successCriteria:[],nextSteps:''},
    cycleAssessHooks:{evidenceToCollect:''},
    studioHandOffIdea:'',
    reflection:''
};
}
function scaffoldPlan(type=state.type){
    const plan = blankPlan(type);
    if(type === 'lesson'){
        plan.objectives = ['Add the main student-friendly learning objective.'];
        plan.materials = ['Add slides, worksheet, text, manipulatives or digital tools.'];
        plan.vocabulary = ['Add key vocabulary.'];
        plan.priorKnowledge = 'Add what students should already know, plus likely misconceptions.';
        plan.lessonFlow = [
            {phase:'Hook',duration:'5 min',teacherRole:'Activate prior knowledge and introduce the lesson purpose.',studentTask:'Respond to a quick question, image, example or problem.',notes:'Keep this short and useful; no circus required.'},
            {phase:'Explicit teaching / modelling',duration:'10 min',teacherRole:'Model the skill, strategy or knowledge clearly. Think aloud where useful.',studentTask:'Listen, notice the steps, answer quick checks.',notes:'Add

```

```

examples or teacher language here.'},
  {phase:'Guided practice',duration:'15 min',teacherRole:'Guide students
through examples and check understanding.',studentTask:'Try the skill with support,
discuss answers, correct mistakes.',notes:'Add mini whiteboard, partner talk or
shared task details.'},
  {phase:'Independent / paired practice',duration:'20
min',teacherRole:'Circulate, question, support and extend.',studentTask:'Complete
the main task independently or with a partner.',notes:'Add what early finishers or
support pupils do.'},
  {phase:'Plenary / exit ticket',duration:'10 min',teacherRole:'Review the
learning and collect quick evidence.',studentTask:'Answer the exit ticket or
explain what they learned.',notes:'Use this evidence for next steps or
CycleAssess.'}
];
plan.differentiation = {
  support:'Add scaffolds, worked examples, sentence stems, reduced load or
adult support.',
  extension:'Add a deeper reasoning, transfer or challenge task.',
  eal:'Add vocabulary support, visuals, sentence frames or language rehearsal.'
};
plan.keyQuestions = ['What do we already know that can help?', 'What strategy
or evidence can we use?', 'How do we know this answer or idea works?'];
plan.assessment = {
  evidence:'Add what you will look/listen for during the lesson.',
  successCriteria:['I can explain the main skill or idea.', 'I can complete the
task using the strategy taught.', 'I can justify my answer with evidence, examples
or reasoning.'],
  nextSteps:'Add likely follow-up teaching based on what students show.'
};
plan.exitTicket = 'Add one quick question or task that shows whether students
met the objective.';
plan.misconceptionCheck = 'Add one likely misunderstanding and how you will
catch it quickly.';
plan.cycleAssessHooks = {evidenceToCollect:'Add the evidence you could later
record in CycleAssess.'};
}else if(type === 'unit'){
  plan.overview = 'Add the big-picture purpose of the unit.';
  plan.drivingQuestion = 'Add the central question or enquiry thread.';
  plan.vocabulary = ['Add recurring unit vocabulary.'];
  const lessonCount = clampInt(getUnitLessonCount(), 1, 24, 6);
  plan.lessonCount = String(lessonCount);
  plan.lessons = Array.from({length:lessonCount},(_,i)=>({title:`Lesson
${i+1}`,duration:'60 min',focus:'Add lesson focus',activities:'Add main learning
activities',assessmentEvidence:'Add what evidence this lesson produces'}));
  plan.differentiation = {support:'Add unit-level support.', extension:'Add
unit-level extension.', eal:'Add language support across the unit.'};
  plan.finalAssessment = 'Add the final task, product, performance or assessment
evidence.';
  plan.cycleAssessHooks = {evidenceToCollect:'Add which outcomes/evidence could
feed CycleAssess.'};

```

```

    }else{
      plan.overview = 'Add the purpose of this scope map.';
      plan.termOrBlock = 'Add term/block name.';
      const weekCount = clampInt(getSequenceWeekCount(), 1, 40, 6);
      plan.weekCount = String(weekCount);
      plan.weeks = Array.from({length:weekCount}, (_,i)=>({week:`Week
${i+1}`, focus:'Add weekly focus', domain:'Add domain', lessons:'Add lessons or
activities', assessment:'Add assessment/evidence'}));
      plan.coverageNotes = 'Add coverage notes, gaps or sequencing decisions.';
      plan.cycleAssessHooks = {coverageNotes:'Add future CycleAssess coverage
link.'};
    }
    return plan;
  }
}

function isBlankText(value){
  return !String(value || '').trim();
}

function inferUnitFocus(title='') {
  const t = String(title || '').toLowerCase();
  if(t.includes('energy')) return 'Compare how France and Singapore source and use
energy, including renewable and non-renewable examples.';
  if(t.includes('water') || t.includes('newater')) return 'Explore how water is
sourced, conserved and recycled, using Singapore and France as comparison points.';
  if(t.includes('food') || t.includes('come from')) return 'Investigate where food
comes from and how local, imported and fair-trade choices affect sustainability.';
  if(t.includes('shadow')) return 'Explore how shadows form and change when light
is blocked by objects.';
  if(t.includes('light')) return 'Build understanding of light sources, light
travel and evidence from simple observations.';
  return title ? `Develop the main ideas from “${title}” through comparison,
discussion and recorded evidence.` : 'Develop one focused step in the unit
sequence.';
}

function inferUnitActivities(title='') {
  const t = String(title || '').toLowerCase();
  if(t.includes('energy')) return 'Sort energy sources into renewable and
non-renewable categories; compare France and Singapore examples using images or
short data cards; complete a comparison table.';
  if(t.includes('water') || t.includes('newater')) return 'Analyse a simple diagram
or photo set showing water supply; discuss conservation choices; create a labelled
diagram or mini poster explaining one solution.';
  if(t.includes('food') || t.includes('come from')) return 'Map where selected
foods come from; sort local/imported/fair-trade examples; discuss food miles and
write a short sustainability reflection.';
  if(t.includes('shadow')) return 'Use torches and objects to test shadow
formation; draw labelled observations; discuss what changes when distance or angle
changes.';
  if(t.includes('light')) return 'Observe light sources; sort examples; complete a
simple investigation or model and record findings with key vocabulary.';
}

```

```

    return 'Introduce the focus with a short stimulus; investigate or compare
examples in pairs/groups; record findings through a table, diagram, short
explanation or exit ticket.';
}
function expectedUnitLessonCount(plan=state.plan){
  if(isCountAiSuggested('unitLesson')){
    return clampInt(concreteSetupValue(plan?.lessonCount) ||
(Array.isArray(plan?.lessons) ? plan.lessons.length : 0), 1, 24, 6);
  }
  return clampInt(plan?.lessonCount || getUnitLessonCount(), 1, 24,
clampInt(getUnitLessonCount(), 1, 24, 6));
}
function expectedSequenceWeekCount(plan=state.plan){
  if(isCountAiSuggested('sequenceWeek')){
    return clampInt(concreteSetupValue(plan?.weekCount) ||
(Array.isArray(plan?.weeks) ? plan.weeks.length : 0), 1, 40, 6);
  }
  return clampInt(plan?.weekCount || getSequenceWeekCount(), 1, 40,
clampInt(getSequenceWeekCount(), 1, 40, 6));
}
function unitPlaceholderLesson(plan, index, total){
  const titleSeed = cleanText(plan?.title || 'this unit');
  const isLast = index + 1 === total;
  return {
    title: isLast ? `Lesson ${index + 1}: Synthesis and final task` : `Lesson
${index + 1}: Complete the unit sequence`,
    duration: cleanText(plan?.duration || '45 min'),
    focus: isLast
      ? `Consolidate the key learning from ${titleSeed} and prepare final
evidence.`
      : `Develop the next key idea in ${titleSeed} so the ${total}-lesson sequence
is complete.`,
    activities: isLast
      ? 'Review the unit vocabulary and key ideas; complete the final diagram,
poster, explanation or reflection; share one piece of evidence.'
      : 'Recall prior learning; investigate or compare a new example; record
findings in a table, diagram or short explanation.',
    assessmentEvidence: isLast
      ? 'Final product, exit ticket, labelled diagram, explanation or teacher
observation linked to the unit outcomes.'
      : 'Completed task, discussion evidence, recorded observations or exit ticket
linked to the lesson focus.'
  };
}
function addQualityWarning(plan, message){
  if(!plan) return;
  if(!Array.isArray(plan._qualityWarnings)) plan._qualityWarnings = [];
  if(!plan._qualityWarnings.includes(message)) plan._qualityWarnings.push(message);
}
function repairUnitLessonRows(plan){

```

```

    if(!Array.isArray(plan.lessons)) plan.lessons = [];
    const targetCount = expectedUnitLessonCount(plan);
    const originalCount = plan.lessons.length;
    if(originalCount < targetCount){
        addQualityWarning(plan, `AI returned ${originalCount} lesson${originalCount === 1 ? '' : 's'}` for a ${targetCount}-lesson unit. Missing lesson cards were added as placeholders.`);
        while(plan.lessons.length < targetCount){
            plan.lessons.push(unitPlaceholderLesson(plan, plan.lessons.length, targetCount));
        }
    }else if(originalCount > targetCount){
        addQualityWarning(plan, `AI returned ${originalCount} lessons for a ${targetCount}-lesson unit. Extra rows were trimmed to match the selected setup.`);
        plan.lessons = plan.lessons.slice(0, targetCount);
    }
    plan.lessonCount = String(targetCount);
    plan.lessons = plan.lessons.map((lesson, index) => {
        const row = {...unitPlaceholderLesson(plan, index, targetCount), ...(lesson || {})};
        row.title = cleanText(row.title || `Lesson ${index + 1}`);
        row.duration = cleanText(row.duration || plan.duration || '45 min');
        row.focus = cleanText(row.focus);
        row.activities = cleanText(row.activities);
        row.assessmentEvidence = cleanText(row.assessmentEvidence);
        if(isBlankText(row.focus)) row.focus = inferUnitFocus(row.title);
        if(isBlankText(row.activities)) row.activities = inferUnitActivities(row.title);
        if(isBlankText(row.assessmentEvidence)) row.assessmentEvidence = 'Exit ticket, completed task, discussion evidence or teacher observation linked to the lesson focus.';
        return row;
    });
}
function repairSequenceWeekRows(plan){
    if(!Array.isArray(plan.weeks)) plan.weeks = [];
    const targetCount = expectedSequenceWeekCount(plan);
    const originalCount = plan.weeks.length;
    if(originalCount < targetCount){
        addQualityWarning(plan, `AI returned ${originalCount} week${originalCount === 1 ? '' : 's'}` for a ${targetCount}-week scope map. Missing week rows were added as placeholders.`);
        while(plan.weeks.length < targetCount){
            plan.weeks.push({week:`Week ${plan.weeks.length + 1}`,focus:'Add a clear weekly learning focus.',domain:plan.domain || 'Domain to confirm',lessons:'Outline the key lessons, tasks or learning activities for this week.',assessment:'Identify evidence to collect during or at the end of the week.'});
        }
    }else if(originalCount > targetCount){
        addQualityWarning(plan, `AI returned ${originalCount} weeks for a

```

```

    ${targetCount}-week scope map. Extra rows were trimmed to match the setup.`);
    plan.weeks = plan.weeks.slice(0, targetCount);
  }
  plan.weekCount = String(targetCount);
  plan.weeks = plan.weeks.map((week, index) => {
    const row = {...(week || {})};
    row.week = String(row.week || `Week ${index + 1}`).trim();
    if(isBlankText(row.focus)) row.focus = 'Add a clear weekly learning focus.';
    if(isBlankText(row.domain)) row.domain = plan.domain || 'Domain to confirm';
    if(isBlankText(row.lessons)) row.lessons = 'Outline the key lessons, tasks or
learning activities for this week.';
    if(isBlankText(row.assessment)) row.assessment = 'Identify evidence to collect
during or at the end of the week.';
    return row;
  });
}

function normalisePlan(input, type=state.type){
  const base = blankPlan(type);
  const plan = {...base, ...(input || {})};
  plan.type = type;
  plan.yearLevel ||= base.yearLevel;
  if(type === 'lesson' && isDurationAiSuggested()){
    plan.duration = cleanText(concreteSetupValue(plan.duration) || '45 min');
  }else{
    plan.duration ||= base.duration;
  }
  plan.learningOutcomes = Array.isArray(plan.learningOutcomes) ?
plan.learningOutcomes : [];
  repairCurriculum(plan);
  if(type === 'lesson'){
    plan.objectives = Array.isArray(plan.objectives) ? plan.objectives : [];
    plan.materials = Array.isArray(plan.materials) ? plan.materials : [];
    plan.vocabulary = Array.isArray(plan.vocabulary) ? plan.vocabulary : [];
    plan.lessonFlow = Array.isArray(plan.lessonFlow) ? plan.lessonFlow : [];
    plan.keyQuestions = Array.isArray(plan.keyQuestions) ? plan.keyQuestions : [];
    plan.misconceptionCheck = typeof plan.misconceptionCheck === 'string' ?
plan.misconceptionCheck : (plan.misconceptions || '');
    plan.differentiation = {...base.differentiation, ...(plan.differentiation ||
{}});
    plan.assessment = {...base.assessment, ...(plan.assessment || {})};
    plan.assessment.successCriteria =
Array.isArray(plan.assessment.successCriteria) ? plan.assessment.successCriteria :
[];
    plan.studioHandOffIdea = typeof plan.studioHandOffIdea === 'string' ?
plan.studioHandOffIdea : (plan.cycleStudioHandOff || plan.cycleStudioBrief || '');
    if(!plan.studioHandOffIdea && /\s*CycleStudio hand-?off
idea:/i.test(plan.reflection || '')){
      plan.studioHandOffIdea = String(plan.reflection).replace(/\s*CycleStudio
hand-?off idea:\s*/i, '').trim();
    }
  }
}

```



```

    plan.reflection = '';
  }
  plan.cycleAssessHooks = {...base.cycleAssessHooks, ...(plan.cycleAssessHooks ||
{}})};
  }
  if(type === 'unit'){
    // Fixed setup selections are the source of truth. When the teacher picks
    // AI suggested, the model's returned count becomes the source of truth.
    if(!isCountAiSuggested('unitLesson')){
      plan.lessonCount = String(clampInt(getUnitLessonCount(), 1, 24, 6));
    }else{
      plan.lessonCount = String(clampInt(concreteSetupValue(plan.lessonCount) ||
(Array.isArray(plan.lessons) ? plan.lessons.length : 0), 1, 24, 6));
    }
    plan.vocabulary = Array.isArray(plan.vocabulary) ? plan.vocabulary : [];
    plan.lessons = Array.isArray(plan.lessons) ? plan.lessons : [];
    repairUnitLessonRows(plan);
    plan.differentiation = {...base.differentiation, ...(plan.differentiation ||
{}})};
    plan.cycleAssessHooks = {...base.cycleAssessHooks, ...(plan.cycleAssessHooks ||
{}})};
  }
  if(type === 'sequence'){
    if(!isCountAiSuggested('sequenceWeek')){
      plan.weekCount = String(clampInt(getSequenceWeekCount(), 1, 40, 6));
    }else{
      plan.weekCount = String(clampInt(concreteSetupValue(plan.weekCount) ||
(Array.isArray(plan.weeks) ? plan.weeks.length : 0), 1, 40, 6));
    }
    plan.weeks = Array.isArray(plan.weeks) ? plan.weeks : [];
    repairSequenceWeekRows(plan);
    plan.cycleAssessHooks = {...base.cycleAssessHooks, ...(plan.cycleAssessHooks ||
{}})};
  }
  return plan;
}

```

/* 5. Navigation + setup */

```

function goTab(tab){
  state.tab = tab;
  qsa('.screen').forEach(screen=>screen.classList.toggle('active', screen.id ===
`screen-${tab}`));
  qsa('.workflow-tab').forEach(btn=>btn.classList.toggle('active', btn.dataset.go
=== tab));
  if(tab === 'create') renderCreateSummary();
  if(tab === 'library') renderLibrary();
  if(tab === 'edit') renderAll();
}
function selectPlanningType(type){
  state.type = type;
}

```

```

    qs('planningTypeSelect').value = type;
    qs('typeChip').innerHTML = planTypeChipHtml(type);
    qs('planTypeDescription').textContent = PLAN_DESCRIPTIONS[type];
    qs('setupTitle').textContent = PLAN_LABELS[type].replace('Scope &
sequence', 'Scope & sequence') + ' setup';
    syncSetupControls(type);
    savePrefs();
    if(state.plan && state.plan.type !== type) state.plan = null;
    renderCreateSummary();
    renderAll();
}
function renderCreateSummary(){
    const curriculumLabel = state.curriculumMode === 'ai_both' ? 'AI chooses all' :
state.curriculumMode === 'manual_domain_ai_lo' ? `${qs('domainSelect').value} + AI
LOs` : `${qs('domainSelect').value} · ${state.selectedOutcomes.length} LOs`;
    qs('createSummary').innerHTML = [
        ['Planning type', PLAN_LABELS[state.type]],
        ['Setup', `${qs('yearLevel').value} · ${getPlanningScaleText()}`],
        ['Curriculum', curriculumLabel]
    ].map(([title, text])=><div
class="summary-card"><b>${esc(title)}</b><span>${esc(text)}</span></div>`).join('')
;
}
function syncCurrentPlanSetup(){
    if(!state.plan) return;
    state.plan.yearLevel = qs('yearLevel').value;
    if(!isDurationAiSuggested()) state.plan.duration = getDurationValue();
    if(state.plan.type === 'unit' && !isCountAiSuggested('unitLesson'))
state.plan.lessonCount = getUnitLessonCount();
    if(state.plan.type === 'sequence' && !isCountAiSuggested('sequenceWeek'))
state.plan.weekCount = getSequenceWeekCount();
    if(!state.plan.context) state.plan.context = qs('extraContextInput').value;
    applyCurriculumToPlan();
}

/* 6. Editor rendering */
function fieldInput(label, path, value='', type='input'){
    const control = type === 'textarea'
        ? <textarea data-path="${path}">${esc(value || '')}</textarea>
        : <input data-path="${path}" value="${esc(value || '')}">;
    return <div class="field"><label>${esc(label)}</label>${control}</div>;
}
function arrayEditor(label, path, items=[]){
    return <div class="field"><label>${esc(label)}</label><textarea
data-array-path="${path}" placeholder="One item per
line">${esc(joinLines(items))}</textarea></div>;
}
function editorCard(title, sub, body, open=true){
    return <details class="editor-card" ${open ? 'open' : ''}>
        <summary class="editor-card-head">

```

```

        <div
class="editor-card-title-wrap"><strong>${esc(title)}</strong><span>${esc(sub ||
'' )}</span></div>
        <span class="editor-card-chevron">v</span>
        </summary>
        <div class="editor-card-body">${body}</div>
    </details>`;
}
function renderEditor(){
    const root = qs('editor');
    if(!state.plan){
        root.innerHTML = '<div class="empty-editor"><strong>No plan yet</strong>Use
Create to start blank, scaffold, template or AI.</div>';
        return;
    }
    qs('workspaceTitle').textContent = ` ${PLAN_LABELS[state.type]} workspace`;
    root.innerHTML = state.type === 'lesson' ? renderLessonEditor(state.plan) :
state.type === 'unit' ? renderUnitEditor(state.plan) :
renderSequenceEditor(state.plan);
}
function renderLessonEditor(p){
    return [
        editorCard('Basics','Title, subject, context and curriculum tags.',
            fieldInput('Title','title',p.title) +
            `<div class="grid-2">${fieldInput('Subject /
strand','subject',p.subject)}${fieldInput('Duration','duration',p.duration)}</div>`
+
            fieldInput('Context','context',p.context,'textarea') +
            arrayEditor('Objectives','objectives',p.objectives) +
            arrayEditor('Materials','materials',p.materials) +
            arrayEditor('Vocabulary','vocabulary',p.vocabulary)
        ),
        editorCard('Prior knowledge','What students already need or may remember.',
fieldInput('Prior knowledge','priorKnowledge',p.priorKnowledge,'textarea')),
        editorCard('Lesson flow','Editable teaching phases.',
renderLessonFlow(p.lessonFlow)),
        editorCard('Differentiation','Support, extension and language support.',
fieldInput('Support','differentiation.support',p.differentiation.support,'textarea'
) +
fieldInput('Extension','differentiation.extension',p.differentiation.extension,'tex
tarea') +
            fieldInput('EAL / language
support','differentiation.eal',p.differentiation.eal,'textarea')
        ),
        editorCard('Light resources','Small things CyclePlanner can sensibly
generate.',
            `<div class="section-actions">
                <button class="small-btn" data-action="key-questions">Generate key

```

```

questions</button>
    <button class="small-btn" data-action="exit-ticket">Generate exit
ticket</button>
    <button class="small-btn" data-action="misconceptions">Generate
misconception check</button>
    </div>` +
    arrayEditor('Key questions','keyQuestions',p.keyQuestions) +
    fieldInput('Exit ticket','exitTicket',p.exitTicket,'textarea') +
    fieldInput('Misconception
check','misconceptionCheck',p.misconceptionCheck,'textarea')
    ),
    editorCard('Assessment','Evidence, success criteria and next steps.',
    fieldInput('Evidence','assessment.evidence',p.assessment.evidence,'textarea'))
+
    arrayEditor('Success
criteria','assessment.successCriteria',p.assessment.successCriteria) +
    fieldInput('Next
steps','assessment.nextSteps',p.assessment.nextSteps,'textarea') +
    fieldInput('CycleAssess evidence
hook','cycleAssessHooks.evidenceToCollect',p.cycleAssessHooks.evidenceToCollect,'te
xtarea')
    ),
    editorCard('CycleStudio hand-off','Worksheet/resource idea to send to
CycleStudio.', fieldInput('CycleStudio hand-off
idea','studioHandOffIdea',p.studioHandOffIdea,'textarea')),
    editorCard('Reflection','Teacher notes after the lesson. Usually blank until
after teaching.', fieldInput('Reflection','reflection',p.reflection,'textarea'))
    ].join('');
}
function renderLessonFlow(rows=[]){
    const empty = !rows.length ? '<div class="empty-editor"
style="padding:18px;margin-bottom:10px"><strong>No lesson phases yet</strong>Start
blank keeps the flow empty. Use planning scaffold if you want Hook / Model / Guided
practice / Independent task / Plenary created for you.</div>' : '';
    const body = rows.map((row,i)=>`
    <div class="editor-card" style="box-shadow:none">
    <div class="editor-card-head"><div><strong>${esc(row.phase || `Phase
${i+1}`)}</strong><span>${esc(row.duration || '')}</span></div><button
class="mini-btn danger" data-remove-row="lessonFlow"
data-index="${i}">Delete</button></div>
    <div
class="grid-2">${fieldInput('Phase`,`lessonFlow.${i}.phase`,row.phase)}${fieldInput
('Duration`,`lessonFlow.${i}.duration`,row.duration)}</div>
    ${fieldInput('Teacher
role`,`lessonFlow.${i}.teacherRole`,row.teacherRole,'textarea')}
    ${fieldInput('Student
task`,`lessonFlow.${i}.studentTask`,row.studentTask,'textarea')}
    ${fieldInput('Notes`,`lessonFlow.${i}.notes`,row.notes,'textarea')}
    </div>` ).join('');
    return empty + body + '<button class="small-btn" data-add-row="lessonFlow">Add

```

```

phase</button>';
}
function renderUnitEditor(p){
  return [
    editorCard('Basics','Overview and shared direction.',
      fieldInput('Title','title',p.title) + `

` +
fieldInput('Overview','overview',p.overview,'textarea') + fieldInput('Driving
question','drivingQuestion',p.drivingQuestion,'textarea') +
arrayEditor('Vocabulary','vocabulary',p.vocabulary)
    ),
    editorCard('Lesson sequence','Each row is one lesson in the unit.',
renderUnitLessons(p.lessons)),
    editorCard('Differentiation','Across the unit.',
fieldInput('Support','differentiation.support',p.differentiation.support,'textarea'
) +
fieldInput('Extension','differentiation.extension',p.differentiation.extension,'tex
tarea') + fieldInput('EAL / language
support','differentiation.eal',p.differentiation.eal,'textarea')),
    editorCard('Assessment','Final assessment and CycleAssess hook.',
fieldInput('Final assessment','finalAssessment',p.finalAssessment,'textarea') +
fieldInput('CycleAssess evidence
hook','cycleAssessHooks.evidenceToCollect',p.cycleAssessHooks.evidenceToCollect,'te
xtarea') + fieldInput('Notes','notes',p.notes,'textarea'))
  ].join('');
}
function renderUnitLessons(rows=[]){
  const empty = !rows.length ? '<div class="empty-editor"
style="padding:18px;margin-bottom:10px"><strong>No unit lessons yet</strong>Start
blank keeps the sequence empty. Use planning scaffold if you want lesson slots
created for you.</div>' : '';
  const body = rows.map((row,i)=>`
    <div class="editor-card" style="box-shadow:none">
      <div class="editor-card-head"><div><strong>${esc(row.title || `Lesson
${i+1}`)}</strong><span>${esc(row.duration || '')}</span></div><button
class="mini-btn danger" data-remove-row="lessons"
data-index="${i}">Delete</button></div>
      <div
class="grid-2">${fieldInput('Title`,`lessons.${i}.title`,row.title)}${fieldInput('D
uration`,`lessons.${i}.duration`,row.duration)}</div>
      ${fieldInput('Focus`,`lessons.${i}.focus`,row.focus,'textarea')}

      ${fieldInput('Activities`,`lessons.${i}.activities`,row.activities,'textarea')}
      ${fieldInput('Assessment
evidence`,`lessons.${i}.assessmentEvidence`,row.assessmentEvidence,'textarea')}
    </div>`).join('');
  return empty + body + '<button class="small-btn" data-add-row="lessons">Add
lesson</button>';
}


```

```

}
function renderSequenceEditor(p){
  return [
    editorCard('Basics','Broad scope map.', fieldInput('Title','title',p.title) +
`<div class="grid-2">${fieldInput('Number of weeks','weekCount',p.weekCount ||
(p.weeks||[]).length || getSequenceWeekCount())}${fieldInput('Term /
block','termOrBlock',p.termOrBlock)}</div>` +
fieldInput('Overview','overview',p.overview,'textarea')),
    editorCard('Sequence map','Week-by-week or block-by-block coverage.',
renderWeeks(p.weeks)),
    editorCard('Notes and coverage','Assessment and coverage hooks.',
fieldInput('Coverage notes','coverageNotes',p.coverageNotes,'textarea') +
fieldInput('CycleAssess coverage
hook','cycleAssessHooks.coverageNotes',p.cycleAssessHooks.coverageNotes,'textarea')
+ fieldInput('Notes','notes',p.notes,'textarea'))
  ].join('');
}
function renderWeeks(rows=[]){
  const empty = !rows.length ? '<div class="empty-editor"
style="padding:18px;margin-bottom:10px"><strong>No sequence blocks
yet</strong>Start blank keeps the map empty. Use planning scaffold if you want six
week/block slots created for you.</div>' : '';
  const body = rows.map((row,i)=>`
    <div class="editor-card" style="box-shadow:none">
      <div class="editor-card-head"><div><strong>${esc(row.week || `Week
${i+1}`)}</strong><span>${esc(row.domain || '')}</span></div><button
class="mini-btn danger" data-remove-row="weeks"
data-index="${i}">Delete</button></div>
      <div class="grid-2">${fieldInput('Week /
block`,`weeks.${i}.week`,row.week)}${fieldInput('Domain`,`weeks.${i}.domain`,row.do
main)}</div>
      ${fieldInput('Focus`,`weeks.${i}.focus`,row.focus,'textarea')}
      ${fieldInput('Lessons /
activities`,`weeks.${i}.lessons`,row.lessons,'textarea')}
      ${fieldInput('Assessment`,`weeks.${i}.assessment`,row.assessment,'textarea')}
    </div>`).join('');
  return empty + body + '<button class="small-btn" data-add-row="weeks">Add week /
block</button>';
}
function addRow(collection){
  if(!state.plan) return;
  const defaults = {
    lessonFlow:{phase:'New
phase',duration:'',teacherRole:'',studentTask:'',notes:''},
    lessons:{title:`Lesson ${((state.plan.lessons||[]).length+1)}`,duration:'60
min',focus:'',activities:'',assessmentEvidence:''},
    weeks:{week:`Week
${((state.plan.weeks||[]).length+1)}`,focus:'',domain:'',lessons:'',assessment:''}
  };
  if(!Array.isArray(state.plan[collection])) state.plan[collection] = [];

```

```

    state.plan[collection].push(defaults[collection]);
    renderAllAndPersist();
}
function removeRow(collection, index){
    if(!state.plan?.[collection]) return;
    state.plan[collection].splice(Number(index),1);
    renderAllAndPersist();
}

/* 7. Preview rendering */
function renderPreview(){
    const root = qs('preview');
    if(!state.plan){
        root.innerHTML = '<div class="preview-page"
style="display:flex;align-items:center;justify-content:center;text-align:center;color:var(--muted)"><div><h3 style="font-size:22px;color:var(--ink)">Plan
preview</h3><p>Create a plan and it will appear here.</p></div></div>';
        return;
    }
    root.innerHTML = state.type === 'lesson' ? lessonPreview(state.plan) : state.type
=== 'unit' ? unitPreview(state.plan) : sequencePreview(state.plan);
}
function previewHeader(p, label){
    return `<div class="preview-head"><div><span
class="kicker">${esc(label)}</span><h1>${esc(p.title || 'Untitled')}</h1><div
class="preview-meta"><span>${esc(p.yearLevel || '')}</span><span>${esc(p.type ===
'unit' ? ((p.lessonCount || (p.lessons||[]).length || '')) + ' lessons') : p.type
=== 'sequence' ? ((p.weekCount || (p.weeks||[]).length || '')) + ' weeks') :
(p.duration || '')}</span></div></div><div
class="preview-badge">CyclePlanner</div></div>`;
}
function curriculumPreview(p){
    return `<div class="print-section accent"><h3>Curriculum
links</h3><p><strong>Domain:</strong> ${esc(p.domain || 'Not
set')}</p>${listHtml(p.learningOutcomes || [])}</div>`;
}
function psec(title, text, accent=false, extraClass=''){
    const classes = ['print-section'];
    if(accent) classes.push('accent');
    if(extraClass) classes.push(extraClass);
    return text ? `<div class="${classes.join('
')}"><h3>${esc(title)}</h3><p>${nl(text)}</p></div>` : '';
}
function lessonPreview(p){
    return `<div class="preview-page">${previewHeader(p, 'Lesson
plan')}${curriculumPreview(p)}${psec('Context',p.context,true)}${(p.objectives||[])
.length?`<div
class="print-section"><h3>Objectives</h3>${listHtml(p.objectives)}</div>`:''}${(p.m
aterials||[]).length?`<div
class="print-section"><h3>Materials</h3>${listHtml(p.materials)}</div>`:''}${psec('

```

```

Prior knowledge',p.priorKnowledge)}<div class="print-section"><h3>Lesson
flow</h3><table><thead><tr><th>Phase</th><th>Time</th><th>Teacher
role</th><th>Student
task</th></tr></thead><tbody>${(p.lessonFlow||[]).map(row=>`<tr><td><strong>${esc(r
ow.phase)}</strong><br>${nl(row.notes)}</td><td>${esc(row.duration)}</td><td>${nl(r
ow.teacherRole)}</td><td>${nl(row.studentTask)}</td></tr>`).join('')}</tbody></tabl
e></div><div
class="two-col">${psec('Support',p.differentiation?.support)}${psec('Extension',p.d
ifferentiation?.extension)}</div>${psec('EAL / language
support',p.differentiation?.eal)}${(p.keyQuestions||[]).length?`<div
class="print-section"><h3>Key
questions</h3>${listHtml(p.keyQuestions)}</div>`:''}${psec('Exit
ticket',p.exitTicket,true)}${psec('Misconception
check',p.misconceptionCheck,true)}${psec('Assessment
evidence',p.assessment?.evidence,true)}${(p.assessment?.successCriteria||[]).length
?`<div class="print-section"><h3>Success
criteria</h3>${listHtml(p.assessment.successCriteria)}</div>`:''}${psec('Next
steps',p.assessment?.nextSteps)}${psec('CycleAssess
hook',p.cycleAssessHooks?.evidenceToCollect)}${psec('CycleStudio hand-off
idea',p.studioHandOffIdea,false,'export-optional')}${psec('Reflection',p.reflection
)}</div>`;
}
function unitLessonCard(row={}, i=0){
  return `
    <article class="unit-lesson-card">
      <div class="unit-lesson-head">
        <div class="unit-lesson-title">${esc(row.title || `Lesson ${i+1}`)}</div>
        <div class="unit-lesson-duration">${esc(row.duration || '')}</div>
      </div>
      <div class="unit-lesson-grid">
        <div class="unit-lesson-block"><div
class="unit-lesson-label">Focus</div><div
class="unit-lesson-body">${compactCell(row.focus,3) || '-'}</div></div>
        <div class="unit-lesson-block"><div
class="unit-lesson-label">Activities</div><div
class="unit-lesson-body">${compactCell(row.activities,4) || '-'}</div></div>
        <div class="unit-lesson-block"><div
class="unit-lesson-label">Evidence</div><div
class="unit-lesson-body">${compactCell(row.assessmentEvidence,3) || '-'}
</div></div>
      </div>
    </article>
  `;
}
function unitLessonCards(lessons=[]){
  if(!lessons.length) return '<p>No lessons added yet.</p>';
  return lessons.map((row,i)=>unitLessonCard(row,i)).join('');
}
function unitLessonExportSections(lessons=[]){
  if(!lessons.length) return '<div class="print-section"><h3>Lesson

```



```

sequence</h3><p>No lessons added yet.</p></div>`;
  return `
```

CP:'CP guidance: keep reading and writing demands short and highly scaffolded. Use oral rehearsal, modelling, manipulatives, images, shared reading, short written responses and frequent checking for understanding.',

CE1:'CE1 guidance: use short independent tasks with clear models, guided practice, partner talk and growing written explanation. Keep instructions explicit and concrete.',

CE2:'CE2 guidance: pupils can handle more structured reasoning, comparison and written explanation, but still need models, vocabulary support and clear steps.'

}[year] || 'Cycle 2 guidance: plan for early-primary learners with explicit modelling, concrete materials, oral rehearsal, guided practice and short independent tasks.';

return `Cycle 2 age guidance for \${year}: \${yearGuidance} Make activities practical and age-appropriate for CP/CE1/CE2. Avoid upper-primary abstraction, long lectures, long essays or mature debate formats unless simplified carefully.`;

}

return 'Cycle 3 age guidance: plan for upper-primary learners with explicit teaching, discussion, structured practice, reasoning, collaboration and increasingly independent written work.';

}

function buildPrompt(extra='') {

const prefs = getPrefs();

const brief = qs('briefInput').value.trim();

const context = [prefs.defaultContext,

qs('extraContextInput').value.trim()].filter(Boolean).join('\n');

const sourceDetails = qs('sourceDetailsInput')?.value.trim() || '';

const sourceSummary = state.sourceFiles.length

? `Attached source files: \${state.sourceFiles.map(file => file.name).join(', ')}.

: 'No source files attached.';

const sourceText = sourcePromptText();

const domain = qs('domainSelect').value;

const schema = blankPlan(state.type);

const usableDomains = availableDomainNames();

const ageGuidance = cyclePromptGuidance();

const scaleRules = state.type === 'lesson'

? (isDurationAiSuggested())

? 'The teacher selected AI suggested lesson length. Choose a realistic lesson duration from the brief, such as 30 min, 45 min, 60 min, 75 min or 90 min. Return that concrete value in duration, not "AI suggested". Make lessonFlow timings add up sensibly to that duration.'

: `Use this exact lesson duration: \${getDurationValue()}. Make lessonFlow timings add up sensibly to that duration.`)

: state.type === 'unit'

? (isCountAiSuggested('unitLesson'))

? 'The teacher selected AI suggested unit length. Choose a realistic number of lessons from the brief, normally 3-10 unless the prompt clearly needs more. Return that concrete number in lessonCount and create exactly that many lesson objects.'

: `Use exactly \${getUnitLessonCount()} lessons. Return lessonCount as

```

"${getUnitLessonCount()}" and create exactly ${getUnitLessonCount()} lesson
objects.`)
    : (isCountAiSuggested('sequenceWeek')
      ? 'The teacher selected AI suggested scope length. Choose a realistic
number of weeks from the brief, normally 3-12 unless the prompt clearly needs a
longer map. Return that concrete number in weekCount and create exactly that many
week rows.'
      : `Use exactly ${getSequenceWeekCount()} weeks. Return weekCount as
"${getSequenceWeekCount()}" and create exactly ${getSequenceWeekCount()} week
rows.`);
    const curriculumRules = state.curriculumMode === 'ai_both'
      ? `Choose one domain exactly from this list: ${usableDomains.join(', ')}
Choose outcomes verbatim from this bank:\n${catalogue()}`
      : state.curriculumMode === 'manual_domain_ai_lo'
      ? `Use this domain exactly: ${domain}. Choose 2-4 outcomes verbatim from this
bank only:\n${catalogue(domain)}`
      : `Use this domain exactly: ${domain}. Use these outcomes
exactly:\n${state.selectedOutcomes.map(x=>'-' + x).join('\n')}`;
    return `You are CyclePlanner, planning for a teacher at an International French
School using ${getCycleLabel()}. Write everything in English. Style: concise,
practical teacher notes.
Planning type: ${state.type}
Year: ${qs('yearLevel').value}
Planning scale: ${getPlanningScaleText()}
Scale rules: ${scaleRules}
Age and cycle fit: ${ageGuidance}
Teacher brief: ${brief || 'No brief provided; create a useful plan.'}
Context: ${context}
Source prompt material: ${sourceSummary}
Source details/instructions: ${sourceDetails || 'None.'}
${sourceText ? `Extracted PPTX slide text:\n${sourceText}` : 'No extracted PPTX
slide text.'}
When source files are attached, use them as planning context or inspiration. Follow
the teacher's source details/instructions, but do not blindly copy unsuitable
material.

```

Curriculum rules:
 \${curriculumRules}

Must include light classroom resources where useful: key questions, exit ticket, misconceptionCheck, assessment evidence. For Cycle 1, these resources should usually be oral, visual, practical, observational or play-based rather than formal written worksheets.

For lesson plans, every lessonFlow row must have useful text in phase, duration, teacherRole and studentTask. Do not leave teacherRole or studentTask blank.

If a lesson plan is based on a unit plan or source plan, align tightly to the matching lesson: objectives, activities, assessment evidence, next steps and CycleAssess hook should clearly connect to that unit sequence.

When a lesson relies on charts, maps, picture cards, handouts or templates, list those resources clearly in Materials. If the resource is not supplied by the

teacher, describe exactly what should be created in studioHandOffIdea instead of pretending it already exists.

For geography lessons comparing water, energy, food, resources or sustainability, avoid oversimplified questions such as "Why doesn't France need NEWater?" or "countries with lots of water". Use precise place-based language: different needs, constraints, infrastructure, resources, scale and management choices.

For unit plans, the lessons array must match the concrete lessonCount you return. No fewer, no more. Every lesson row must include title, duration, focus, activities and assessmentEvidence. Do not leave focus or activities blank. Keep lesson-sequence cells print-friendly: focus under 22 words, activities as 2-3 concise classroom actions, and evidence under 25 words. Avoid long paragraphs inside table cells. Do not use numbered-list text such as "1. ... 2. ... 3. ..." inside activities; use short sentence fragments separated by semicolons or new lines.

For scope and sequence plans, the weeks array must match the concrete weekCount you return. Every week row must include week, focus, domain, lessons and assessment. If the topic is scientific, choose the science/world-discovery domain available in the selected cycle, not literacy. Always return a non-empty domain and 2-4 exact learningOutcomes copied from the allowed bank when curriculum mode lets AI choose them.

Bigger worksheets and resource packs should be described only in the studioHandOffIdea field. Do not put CycleStudio hand-off ideas inside reflection. Leave reflection blank unless it contains actual after-teaching reflection notes. `${extra}`

Return ONLY valid JSON matching this schema:

```
${JSON.stringify(schema,null,2)}`;  
}
```

```
async function callAI(prompt, options={}){  
  const prefs = getPrefs();  
  const key = (qs('apiKey').value || prefs.apiKey || '').trim();  
  const model = (qs('modelName').value || prefs.modelName ||  
'claude-opus-4-5').trim();  
  if(!key){  
    goTab('settings');  
    throw new Error('Add your Anthropic API key in Settings first.');  }  
  const content = [];  
  const includeSources = !!options.includeSources;  
  if(includeSources){  
    state.sourceFiles.forEach(file=>{  
      if(file.kind === 'pdf'){  
  
content.push({type:'document',source:{type:'base64',media_type:file.mimeType ||  
'application/pdf',data:file.base64}});  
      }else if(file.kind === 'image'){  
        content.push({type:'image',source:{type:'base64',media_type:file.mimeType  
|| 'image/png',data:file.base64}});  
      }  
    });  
  }  
}
```

```

}
content.push({type:'text',text:prompt});
const headers = {
  'Content-Type':'application/json',
  'x-api-key':key,
  'anthropic-version':'2023-06-01',
  'anthropic-dangerous-direct-browser-access':'true'
};
if(includeSources && state.sourceFiles.some(file => file.kind === 'pdf'))
headers['anthropic-beta'] = 'pdfs-2024-09-25';
const response = await fetch('https://api.anthropic.com/v1/messages', {
  method:'POST',
  headers,
  body:JSON.stringify({
    model,
    max_tokens:5000,
    messages:[{role:'user',content}]
  })
});
const data = await response.json().catch(()=>null);
if(!response.ok || data?.error) throw new Error(data?.error?.message || 'API
error ' + response.status);
const raw = (data.content || []).map(block=>block.text ||
'').join('').trim().replace(/```json|```/g,'').trim();
try{return JSON.parse(raw);}
catch(err){console.error(raw);throw new Error('AI returned invalid JSON. Tiny
tragedy, fixable.');}
}
async function generatePlan(){
  clearStatus();
  const usableDomains = availableDomainNames();
  if(!usableDomains.length){
    setStatus(`${getCycleLabel()} does not have an available curriculum bank yet.`,
'warn');
    return;
  }
  if(state.curriculumMode !== 'ai_both' && !selectedDomainHasOutcomes()){
    setStatus(`${qs('domainSelect').value || 'This domain'} is not available yet.
Choose another domain.`, 'warn');
    return;
  }
  if(state.curriculumMode === 'manual_both' && !state.selectedOutcomes.length){
    setStatus('Choose at least one learning outcome before generating.', 'warn');
    return;
  }
  savePrefs();
  const generationLabel = getGenerationLabel();
  setCreateBusy(true, `Generating your ${generationLabel}...`, 'Units and scope maps
can take 20-60 seconds. Nothing is frozen; the robot is just chewing.');
```

```

  setStatus(`Generating your ${generationLabel}... this can take 20-60 seconds.`,

```

```

'warn');
setBusy(true, 'Generating plan...', 'Building a practical planning document.');
```

```

try{
  const result = await callAI(buildPrompt(), {includeSources:true});
  state.plan = normalisePlan(result, state.type);
  saveCurrent('history', true);
  renderAll();
  goTab('edit');
  if(state.plan?._qualityWarnings?.length){
    setStatus(state.plan._qualityWarnings[0], 'warn', 'editStatusBox');
  }else{
    setStatus(`${PLAN_LABELS[state.type]} generated.`, 'ok', 'editStatusBox');
  }
}catch(err){
  const message = err?.message || 'Generation failed.';
  setStatus(message, 'err');
}finally{
  setCreateBusy(false);
  setBusy(false);
}
}

async function transformPlan(mode){
  if(!state.plan) return setStatus('Create a plan first.', 'warn', 'editStatusBox');
  const request = mode === 'formal'
    ? 'Rewrite this planning document in a more formal school-planning style while keeping it useful and not bloated.'
    : 'Rewrite this planning document into a quicker, leaner teacher-use version. Preserve the key teaching sequence and assessment evidence.';
  setBusy(true, mode === 'formal' ? 'Making it formal...' : 'Making it quick...', 'Keeping structure, changing depth and tone.');
```

```

  try{
    const result = await callAI(`You are improving an existing CyclePlanner ${state.plan.type}. ${request}\nReturn ONLY valid JSON using the same schema.\nExisting plan JSON:\n${JSON.stringify(state.plan, null, 2)}`);
    state.plan = normalisePlan(result, state.type);
    saveCurrent('history', true);
    renderAll();
    setStatus(mode === 'formal' ? 'Formal version ready.' : 'Quick version ready.', 'ok', 'editStatusBox');
  }catch(err){
    setStatus(err.message || 'Transformation failed.', 'err', 'editStatusBox');
  }finally{setBusy(false);}
}

async function modifyWithAI(){
  if(!state.plan) return setStatus('Create a plan first.', 'warn', 'editStatusBox');
  const request = qs('modifyRequest').value.trim();
  if(!request) return setStatus('Write what you want changed first.', 'warn', 'editStatusBox');
  setBusy(true, 'Modifying with AI...', 'Applying requested changes while preserving the rest of the plan.');
```

```
    try{
      const result = await callAI(`You are modifying an existing CyclePlanner
${state.plan.type}.
```

```
Teacher request:
${request}
```

Rules:

- Change only the parts the teacher asked to change.
- Preserve the current planning type and JSON schema.
- Keep the plan concise and practical.
- Keep Cycle 3 domain and learning outcomes valid; do not invent outcomes.
- Return ONLY valid JSON using the same schema.

Existing plan JSON:

```
${JSON.stringify(state.plan,null,2)}`);
  state.plan = normalisePlan(result,state.type);
  saveCurrent('history',true);
  renderAll();
  setStatus('AI modification applied.','ok','editStatusBox');
}catch(err){
  setStatus(err.message || 'AI modification failed.','err','editStatusBox');
}finally{setBusy(false);}
}
async function lightResource(kind){
  if(!state.plan) return setStatus('Create a plan first.','warn','editStatusBox');
  const instructions = {
    key_questions:'Add or improve practical key questions for discussion and
checking understanding.',
    exit_ticket:'Add or improve a short classroom-ready exit ticket.',
    misconceptions:'Add or improve the misconceptionCheck field with likely student
misunderstandings and a fast way to catch them.'
  };
  setBusy(true,'Adding light resource...','Small resource only. CycleStudio handles
bigger worksheet work.');
```

```
    try{
      const result = await callAI(`Improve this CyclePlanner ${state.plan.type}.
${instructions[kind]} Keep the plan concise. Return ONLY valid JSON using the same
schema.\n${JSON.stringify(state.plan,null,2)}`);
      state.plan = normalisePlan(result,state.type);
      saveCurrent('history',true);
      renderAll();
      setStatus('Light resource added.','ok','editStatusBox');
    }catch(err){
      setStatus(err.message || 'Could not add resource.','err','editStatusBox');
    }finally{setBusy(false);}
  }
}
```

```
/* 9. Library + settings */
```

```
function saveCurrent(bucket='history', silent=false){
```

```

    if(!state.plan){
        state.plan = blankPlan(state.type);
        syncCurrentPlanSetup();
    }
    const key = bucket === 'template' ? STORAGE.templates : STORAGE.history;
    const list = readJSON(key, []);
    list.unshift({id:makeId(),createdAt:new
Date().toISOString(),type:state.plan.type,title:state.plan.title ||
'Untitled',plan:clonePlain(state.plan)});
    writeJSON(key, list.slice(0,60));
    writeJSON(STORAGE.current, state.plan);
    if(!silent) setStatus(bucket === 'template' ? 'Saved as template.' : 'Saved to
history.', 'ok','editStatusBox');
}
function renderLibrary(){
    renderSavedList(STORAGE.history,'historyList');
    renderSavedList(STORAGE.templates,'templateList');
}
function renderSavedList(key, rootId){
    const items = readJSON(key, []);
    qs(rootId).innerHTML = items.length
        ? items.map(item=>`<div
class="saved-item"><div><strong>${esc(item.title)}</strong><span>${esc(PLAN_LABELS[
item.type] || item.type)} · ${new
Date(item.createdAt).toLocaleString()}</span></div><div class="pill-row"><button
class="mini-btn" data-load-key="${key}"
data-load-id="${item.id}">Load</button><button class="mini-btn danger"
data-delete-key="${key}"
data-delete-id="${item.id}">Delete</button></div></div>`).join('')
        : '<p class="tiny">Nothing saved yet.</p>';
}
function loadSaved(key,id){
    const item = readJSON(key,[]).find(entry=>entry.id === id);
    if(!item) return;
    state.type = item.type;
    state.plan = normalisePlan(clonePlain(item.plan), state.type);
    selectPlanningType(state.type);
    goTab('edit');
    renderAll();
    setStatus('Loaded: ' + item.title, 'ok','editStatusBox');
}
function deleteSaved(key,id){
    writeJSON(key, readJSON(key,[]).filter(item=>item.id !== id));
    renderLibrary();
    renderTemplateModal();
}
function clearLibrary(){
    if(confirm('Clear all saved history and templates?')){
        writeJSON(STORAGE.history,[]);
        writeJSON(STORAGE.templates,[]);
    }
}

```



```

    renderLibrary();
  }
}
function openTemplates(){
  renderTemplateModal();
  qs('templateModal').classList.add('open');
}
function closeTemplates(){
  qs('templateModal').classList.remove('open');
}
function renderTemplateModal(){
  const items = readJSON(STORAGE.templates, []).filter(item=>item.type ===
state.type);
  qs('templateModalList').innerHTML = items.length
    ? items.map(item=>`<div
class="saved-item"><div><strong>${esc(item.title)}</strong><span>${new
Date(item.createdAt).toLocaleDateString()}</span></div><button class="mini-btn"
data-template-id="${item.id}">Use</button></div>`).join('')
    : '<p class="tiny">No templates for this planning type yet.</p>';
}
function useTemplate(id){
  const item = readJSON(STORAGE.templates, []).find(entry=>entry.id === id);
  if(!item) return;
  state.type = item.type;
  state.plan = normalisePlan(clonePlain(item.plan), state.type);
  selectPlanningType(state.type);
  closeTemplates();
  goTab('edit');
  renderAll();
  setStatus('Template loaded.', 'ok', 'editStatusBox');
}
function saveSettings(){
  savePrefs();
  setStatus('Settings saved.', 'ok', 'editStatusBox');
}
function copyText(){
  if(!state.plan) return setStatus('Nothing to copy yet.', 'warn', 'editStatusBox');

  navigator.clipboard?.writeText(JSON.stringify(state.plan, null, 2)).then(()=>setStatu
s('Copied.', 'ok', 'editStatusBox')).catch(()=>setStatus('Could not
copy.', 'warn', 'editStatusBox'));
}
function copyStudioBrief(){
  const p = state.plan;
  if(!p) return setStatus('Create a plan first.', 'warn', 'editStatusBox');
  const brief = `Create a CycleStudio worksheet/resource to support this ${p.type}:
Title: ${p.title}
Year: ${p.yearLevel}
Domain: ${p.domain || ''}
Learning outcomes:

```

```
`${p.learningOutcomes || []}.map(o=>' - '+o).join('\n')}`
```

Context:

```
`${p.context || p.overview || ''}`
```

Create a short classroom resource, exit ticket, practice task, or worksheet aligned to the plan. Keep it printable and age-appropriate.`;

```
navigator.clipboard?.writeText(brief).then(()=>setStatus('CycleStudio brief copied.', 'ok', 'editStatusBox')).catch(()=>setStatus('Could not copy CycleStudio brief.', 'warn', 'editStatusBox'));
```

```
}
```

```
function safeFilename(name='CyclePlanner_plan'){  
  return String(name || 'CyclePlanner_plan')  
    .trim()  
    .replace(/[^\a-z0-9\-\_]/gi, '_')  
    .replace(/_+/g, '_')  
    .replace(/^_|_$/g, '') || 'CyclePlanner_plan';  
}
```

```
function planQualityIssues(plan=state.plan){  
  const issues = [];  
  if(!plan) return issues;  
  if(plan.type === 'unit'){  
    const declared = clampInt(plan.lessonCount || getUnitLessonCount(), 1, 24, 0);  
    const actual = Array.isArray(plan.lessons) ? plan.lessons.length : 0;  
    if(declared && actual !== declared){  
      issues.push(`This unit says ${declared} lesson${declared === 1 ? '' : 's'}  
but only has ${actual}. Add missing lesson cards or adjust the setup before  
exporting.`);  
    }  
    (plan.lessons || []).forEach((lesson, index) => {  
      if(isBlankText(lesson.focus) || isBlankText(lesson.activities) ||  
isBlankText(lesson.assessmentEvidence)){  
        issues.push(`Lesson ${index + 1} is missing focus, activities or  
evidence.`);  
      }  
    });  
  }  
  if(plan.type === 'sequence'){  
    const declared = clampInt(plan.weekCount || getSequenceWeekCount(), 1, 40, 0);  
    const actual = Array.isArray(plan.weeks) ? plan.weeks.length : 0;  
    if(declared && actual !== declared){  
      issues.push(`This scope map says ${declared} week${declared === 1 ? '' : 's'}  
but only has ${actual}. Add missing week rows or adjust the setup before  
exporting.`);  
    }  
  }  
  return issues;  
}
```

```

function showPlanQualityWarnings(target='editStatusBox'){
  const issues = planQualityIssues(state.plan);
  if(issues.length){
    setStatus(issues[0], 'err', target);
    return false;
  }
  return true;
}
/* Text-based PDF export
-----
This avoids html2canvas. html2canvas makes a screenshot PDF, which
looks tidy but turns text into pixels. This exporter draws real PDF
text with jsPDF, so the exported file stays searchable, selectable
and editable in PDF tools. */
const PDF_TEXT_EXPORT = Object.freeze({
  w:210,
  h:297,
  margin:15,
  contentW:180,
  colours:{
    ink:[16,40,71],
    muted:[104,117,138],
    crimson:[159,18,57],
    line:[224,229,238],
    soft:[248,241,242],
    blueSoft:[244,246,250]
  }
});
function pdfPlain(value='') {
  return cleanText(value)
    .replace(/\u2013\u2014/g, '-')
    .replace(/\u2018\u2019/g, '"')
    .replace(/\u201c\u201d/g, '"')
    .replace(/\u00a0/g, ' ');
}
function pdfLineHeight(size=10, factor=1.28){ return size * 0.3528 * factor; }
function pdfSetFont(pdf, size=10, style='normal',
  colour=PDF_TEXT_EXPORT.colours.ink){
  pdf.setFont('helvetica', style);
  pdf.setFontSize(size);
  pdf.setTextColor(colour[0], colour[1], colour[2]);
}
function pdfEnsureSpace(pdf, cursor, needed=8){
  if(cursor.y + needed <= PDF_TEXT_EXPORT.h - PDF_TEXT_EXPORT.margin) return;
  pdf.addPage();
  cursor.y = PDF_TEXT_EXPORT.margin + 5;
}
function pdfWriteWrapped(pdf, cursor, text, opts={}){
  const value = pdfPlain(text);
  if(!value) return;

```

```

const x = opts.x ?? PDF_TEXT_EXPORT.margin;
const width = opts.width ?? PDF_TEXT_EXPORT.contentW;
const size = opts.size ?? 10;
const style = opts.style || 'normal';
const colour = opts.colour || PDF_TEXT_EXPORT.colours.ink;
const lineH = pdfLineHeight(size, opts.lineFactor || 1.28);
const paragraphs = value.split('\n');
pdfSetFont(pdf, size, style, colour);
paragraphs.forEach((para, paraIndex) => {
  const cleanPara = para.trim();
  if(!cleanPara){ cursor.y += lineH * .65; return; }
  const lines = pdf.splitTextToSize(cleanPara, width);
  lines.forEach(line => {
    pdfEnsureSpace(pdf, cursor, lineH + 1);
    pdf.text(line, x, cursor.y);
    cursor.y += lineH;
  });
  if(paraIndex < paragraphs.length - 1) cursor.y += lineH * .25;
});
cursor.y += opts.gapAfter ?? 2.4;
}
function pdfWriteBulletList(pdf, cursor, items=[], opts={}){
  const list = (items || []).map(pdfPlain).filter(Boolean);
  if(!list.length) return;
  const x = opts.x ?? PDF_TEXT_EXPORT.margin;
  const width = opts.width ?? PDF_TEXT_EXPORT.contentW;
  const size = opts.size ?? 10;
  const lineH = pdfLineHeight(size, 1.24);
  pdfSetFont(pdf, size, 'normal', opts.colour || PDF_TEXT_EXPORT.colours.ink);
  list.forEach(item => {
    const lines = pdf.splitTextToSize(item, width - 6);
    lines.forEach((line, index) => {
      pdfEnsureSpace(pdf, cursor, lineH + 1);
      if(index === 0) pdf.text('-', x, cursor.y);
      pdf.text(line, x + 5, cursor.y);
      cursor.y += lineH;
    });
    cursor.y += 1.2;
  });
  cursor.y += opts.gapAfter ?? 2;
}
function pdfSection(pdf, cursor, title, bodyFn, opts={}){
  pdfEnsureSpace(pdf, cursor, 13);
  cursor.y += opts.before ?? 1.5;
  const x = PDF_TEXT_EXPORT.margin;
  const w = PDF_TEXT_EXPORT.contentW;
  if(opts.accent){
    pdf.setFillColor(...PDF_TEXT_EXPORT.colours.soft);
    pdf.setDrawColor(...PDF_TEXT_EXPORT.colours.line);
    if(typeof pdf.roundedRect === 'function') pdf.roundedRect(x - 2, cursor.y - 6,

```

```

w + 4, 10, 2, 2, 'FD');
    else pdf.rect(x - 2, cursor.y - 6, w + 4, 10, 'FD');
  }
  pdfSetFont(pdf, 12, 'bold', opts.accent ? PDF_TEXT_EXPORT.colours.crimson :
PDF_TEXT_EXPORT.colours.ink);
  pdf.text(pdfPlain(title), x, cursor.y);
  cursor.y += 5;
  pdf.setDrawColor(...PDF_TEXT_EXPORT.colours.line);
  pdf.line(x, cursor.y, x + w, cursor.y);
  cursor.y += 5;
  bodyFn();
  cursor.y += 3;
}
function pdfKeyValue(pdf, cursor, label, value){
  const clean = pdfPlain(value);
  if(clean) pdfWriteWrapped(pdf, cursor, `${label}: ${clean}`, {size:10,
gapAfter:1.8});
}
function pdfHeader(pdf, cursor, plan, label){
  const m = PDF_TEXT_EXPORT.margin;
  pdf.setFillColor(...PDF_TEXT_EXPORT.colours.blueSoft);
  pdf.rect(0, 0, PDF_TEXT_EXPORT.w, 33, 'F');
  pdfSetFont(pdf, 9, 'bold', PDF_TEXT_EXPORT.colours.crimson);
  pdf.text(pdfPlain(label).toUpperCase(), m, 13);
  pdfSetFont(pdf, 20, 'bold', PDF_TEXT_EXPORT.colours.ink);
  const titleLines = pdf.splitTextToSize(pdfPlain(plan.title || 'Untitled'),
PDF_TEXT_EXPORT.contentW - 34);
  pdf.text(titleLines.slice(0, 2), m, 22, {lineHeightFactor:1.05});
  pdfSetFont(pdf, 9, 'bold', PDF_TEXT_EXPORT.colours.crimson);
  pdf.text('CyclePlanner', PDF_TEXT_EXPORT.w - m - 26, 13);
  cursor.y = 43 + Math.max(0, titleLines.length - 1) * 5;
  const scale = plan.type === 'unit'
    ? `${plan.lessonCount || (plan.lessons || []).length || ''} lessons`
    : plan.type === 'sequence'
    ? `${plan.weekCount || (plan.weeks || []).length || ''} weeks`
    : (plan.duration || '');
  pdfSetFont(pdf, 10, 'normal', PDF_TEXT_EXPORT.colours.muted);
  pdf.text(pdfPlain([plan.yearLevel, scale].filter(Boolean).join(' · ')), m,
cursor.y);
  cursor.y += 9;
}
function pdfCurriculum(pdf, cursor, plan){
  pdfSection(pdf, cursor, 'Curriculum links', () => {
    pdfKeyValue(pdf, cursor, 'Domain', plan.domain || 'Not set');
    pdfWriteBulletList(pdf, cursor, plan.learningOutcomes || []);
  }, {accent:true});
}
function pdfTextSection(pdf, cursor, title, text, accent=false){
  if(pdfPlain(text)) pdfSection(pdf, cursor, title, () => pdfWriteWrapped(pdf,
cursor, text, {size:10}), {accent});
}

```

```

}
function pdfArraySection(pdf, cursor, title, items, accent=false){
  if((items || []).filter(Boolean).length) pdfSection(pdf, cursor, title, () =>
pdfWriteBulletList(pdf, cursor, items), {accent});
}
function pdfDrawLessonFlowTable(pdf, cursor, rows=[]){
  const tableX = PDF_TEXT_EXPORT.margin;
  const tableW = PDF_TEXT_EXPORT.contentW;
  const cols = [
    {label:'Phase', width:36},
    {label:'Time', width:16},
    {label:'Teacher role', width:82},
    {label:'Student task', width:46}
  ];
  const padding = 2.6;
  const headerH = 10;
  const rowGap = 1.1;

  const drawHeader = () => {
    pdfEnsureSpace(pdf, cursor, headerH + 6);
    let x = tableX;
    pdf.setFillColor(...PDF_TEXT_EXPORT.colours.soft);
    pdf.setDrawColor(...PDF_TEXT_EXPORT.colours.line);
    pdf.rect(tableX, cursor.y, tableW, headerH, 'FD');
    cols.forEach(col => {
      pdf.line(x, cursor.y, x, cursor.y + headerH);
      pdfSetFont(pdf, 9.2, 'bold', PDF_TEXT_EXPORT.colours.ink);
      pdf.text(col.label, x + padding, cursor.y + 6.5);
      x += col.width;
    });
    pdf.line(tableX + tableW, cursor.y, tableX + tableW, cursor.y + headerH);
    cursor.y += headerH;
  };

  drawHeader();
  rows.forEach(row => {
    const cells = [
      [pdfPlain(row.phase || 'Phase'), pdfPlain(row.notes ||
'')].filter(Boolean).join('\n'),
      pdfPlain(row.duration || ''),
      pdfPlain(row.teacherRole || ''),
      pdfPlain(row.studentTask || '')
    ];
    const wrapped = cells.map((text, index) => {
      pdfSetFont(pdf, index === 0 ? 8.8 : 8.6, index === 0 ? 'bold' : 'normal',
PDF_TEXT_EXPORT.colours.ink);
      return pdf.splitTextToSize(text || ' ', cols[index].width - padding * 2);
    });
    const rowH = Math.max(14, ...wrapped.map(lines => lines.length *
pdfLineHeight(8.6, 1.22) + padding * 2));
  });
}

```

```

    if(cursor.y + rowH > PDF_TEXT_EXPORT.h - PDF_TEXT_EXPORT.margin){
      pdf.addPage();
      cursor.y = PDF_TEXT_EXPORT.margin + 5;
      drawHeader();
    }
    let x = tableX;
    pdf.setDrawColor(...PDF_TEXT_EXPORT.colours.line);
    pdf.rect(tableX, cursor.y, tableW, rowH);
    cols.forEach((col, index) => {
      pdf.line(x, cursor.y, x, cursor.y + rowH);
      pdfSetFont(pdf, index === 0 ? 8.8 : 8.6, index === 0 ? 'bold' : 'normal',
PDF_TEXT_EXPORT.colours.ink);
      pdf.text(wrapped[index], x + padding, cursor.y + padding + 3.2,
{lineHeightFactor:1.22});
      x += col.width;
    });
    pdf.line(tableX + tableW, cursor.y, tableX + tableW, cursor.y + rowH);
    cursor.y += rowH + rowGap;
  });
  cursor.y += 2;
}
function pdfDrawSequenceMapTable(pdf, cursor, rows=[]){
  const tableX = PDF_TEXT_EXPORT.margin;
  const tableW = PDF_TEXT_EXPORT.contentW;
  const cols = [
    {label:'Week', width:18},
    {label:'Focus', width:40},
    {label:'Domain', width:38},
    {label:'Lessons / activities', width:58},
    {label:'Assessment', width:26}
  ];
  const padding = 2.4;
  const headerH = 10;
  const rowGap = 1.1;

  const drawHeader = () => {
    pdfEnsureSpace(pdf, cursor, headerH + 6);
    let x = tableX;
    pdf.setFillColors(...PDF_TEXT_EXPORT.colours.soft);
    pdf.setDrawColor(...PDF_TEXT_EXPORT.colours.line);
    pdf.rect(tableX, cursor.y, tableW, headerH, 'FD');
    cols.forEach(col => {
      pdf.line(x, cursor.y, x, cursor.y + headerH);
      pdfSetFont(pdf, 8.8, 'bold', PDF_TEXT_EXPORT.colours.ink);
      pdf.text(col.label, x + padding, cursor.y + 6.5);
      x += col.width;
    });
    pdf.line(tableX + tableW, cursor.y, tableX + tableW, cursor.y + headerH);
    cursor.y += headerH;
  };

```

```

drawHeader();
rows.forEach((row, index) => {
  const cells = [
    pdfPlain(row.week || String(index + 1)),
    pdfPlain(row.focus || ''),
    pdfPlain(row.domain || ''),
    pdfPlain(row.lessons || ''),
    pdfPlain(row.assessment || '')
  ];
  const wrapped = cells.map((text, cellIndex) => {
    pdfSetFont(pdf, cellIndex === 0 ? 8.8 : 8.4, cellIndex === 0 ? 'bold' :
'normal', PDF_TEXT_EXPORT.colours.ink);
    return pdf.splitTextToSize(text || ' ', cols[cellIndex].width - padding * 2);
  });
  const rowH = Math.max(14, ...wrapped.map(lines => lines.length *
pdfLineHeight(8.4, 1.2) + padding * 2));
  if(cursor.y + rowH > PDF_TEXT_EXPORT.h - PDF_TEXT_EXPORT.margin){
    pdf.addPage();
    cursor.y = PDF_TEXT_EXPORT.margin + 5;
    drawHeader();
  }
  let x = tableX;
  pdf.setDrawColor(...PDF_TEXT_EXPORT.colours.line);
  pdf.rect(tableX, cursor.y, tableW, rowH);
  cols.forEach((col, cellIndex) => {
    pdf.line(x, cursor.y, x, cursor.y + rowH);
    pdfSetFont(pdf, cellIndex === 0 ? 8.8 : 8.4, cellIndex === 0 ? 'bold' :
'normal', PDF_TEXT_EXPORT.colours.ink);
    pdf.text(wrapped[cellIndex], x + padding, cursor.y + padding + 3.1,
{lineHeightFactor:1.2});
    x += col.width;
  });
  pdf.line(tableX + tableW, cursor.y, tableX + tableW, cursor.y + rowH);
  cursor.y += rowH + rowGap;
});
cursor.y += 2;
}
function pdfLessonPlan(pdf, cursor, plan){
  pdfHeader(pdf, cursor, plan, 'Lesson plan');
  pdfCurriculum(pdf, cursor, plan);
  pdfTextSection(pdf, cursor, 'Context', plan.context, true);
  pdfArraySection(pdf, cursor, 'Objectives', plan.objectives);
  pdfArraySection(pdf, cursor, 'Materials', plan.materials);
  pdfArraySection(pdf, cursor, 'Vocabulary', plan.vocabulary);
  pdfTextSection(pdf, cursor, 'Prior knowledge', plan.priorKnowledge);
  if((plan.lessonFlow || []).length){
    pdfSection(pdf, cursor, 'Lesson flow', () => pdfDrawLessonFlowTable(pdf,
cursor, plan.lessonFlow || []));
  }
}

```



```

pdfTextSection(pdf, cursor, 'Support', plan.differentiation?.support);
pdfTextSection(pdf, cursor, 'Extension', plan.differentiation?.extension);
pdfTextSection(pdf, cursor, 'EAL / language support', plan.differentiation?.eal);
pdfArraySection(pdf, cursor, 'Key questions', plan.keyQuestions);
pdfTextSection(pdf, cursor, 'Exit ticket', plan.exitTicket, true);
pdfTextSection(pdf, cursor, 'Misconception check', plan.misconceptionCheck,
true);
pdfTextSection(pdf, cursor, 'Assessment evidence', plan.assessment?.evidence,
true);
pdfArraySection(pdf, cursor, 'Success criteria',
plan.assessment?.successCriteria);
pdfTextSection(pdf, cursor, 'Next steps', plan.assessment?.nextSteps);
pdfTextSection(pdf, cursor, 'CycleAssess hook',
plan.cycleAssessHooks?.evidenceToCollect);
pdfTextSection(pdf, cursor, 'Reflection', plan.reflection);
}
function pdfUnitPlan(pdf, cursor, plan){
pdfHeader(pdf, cursor, plan, 'Unit plan');
pdfCurriculum(pdf, cursor, plan);
if(Array.isArray(plan._qualityWarnings) && plan._qualityWarnings.length){
pdfSection(pdf, cursor, 'Pre-export quality note', () =>
pdfWriteBulletList(pdf, cursor, plan._qualityWarnings), {accent:true});
}
pdfTextSection(pdf, cursor, 'Overview', plan.overview, true);
pdfTextSection(pdf, cursor, 'Driving question', plan.drivingQuestion);
pdfArraySection(pdf, cursor, 'Vocabulary', plan.vocabulary);
if((plan.lessons || []).length){
pdfSection(pdf, cursor, 'Lesson sequence', () => {
(plan.lessons || []).forEach((row, index) => {
pdfEnsureSpace(pdf, cursor, 24);
pdf.setFillColor(...PDF_TEXT_EXPORT.colours.blueSoft);
pdf.setDrawColor(...PDF_TEXT_EXPORT.colours.line);
pdf.rect(PDF_TEXT_EXPORT.margin - 1, cursor.y - 5, PDF_TEXT_EXPORT.contentW
+ 2, 9, 'FD');
pdfSetFont(pdf, 10.5, 'bold', PDF_TEXT_EXPORT.colours.ink);
pdf.text(pdfPlain(`${index + 1}. ${row.title || 'Lesson'}${row.duration ? '
' + row.duration : ''}`), PDF_TEXT_EXPORT.margin + 1, cursor.y + 1);
cursor.y += 9;
pdfWriteWrapped(pdf, cursor, `Focus: ${row.focus || ''}`,
{x:PDF_TEXT_EXPORT.margin + 3, width:PDF_TEXT_EXPORT.contentW - 3, size:9.6,
gapAfter:1});
pdfWriteWrapped(pdf, cursor, `Activities: ${row.activities || ''}`,
{x:PDF_TEXT_EXPORT.margin + 3, width:PDF_TEXT_EXPORT.contentW - 3, size:9.6,
gapAfter:1});
pdfWriteWrapped(pdf, cursor, `Evidence: ${row.assessmentEvidence || ''}`,
{x:PDF_TEXT_EXPORT.margin + 3, width:PDF_TEXT_EXPORT.contentW - 3, size:9.4,
colour:PDF_TEXT_EXPORT.colours.muted, gapAfter:1});
cursor.y += 3;
});
});
});
}

```

```

    }
    pdfTextSection(pdf, cursor, 'Support', plan.differentiation?.support);
    pdfTextSection(pdf, cursor, 'Extension', plan.differentiation?.extension);
    pdfTextSection(pdf, cursor, 'EAL / language support', plan.differentiation?.eal);
    pdfTextSection(pdf, cursor, 'Final assessment', plan.finalAssessment, true);
    pdfTextSection(pdf, cursor, 'CycleAssess hook',
plan.cycleAssessHooks?.evidenceToCollect);
    pdfTextSection(pdf, cursor, 'Notes', plan.notes);
  }
function pdfSequencePlan(pdf, cursor, plan){
  pdfHeader(pdf, cursor, plan, 'Scope & sequence');
  pdfTextSection(pdf, cursor, 'Overview', plan.overview, true);
  pdfTextSection(pdf, cursor, 'Term / block', plan.termOrBlock);
  if((plan.weeks || []).length){
    pdfSection(pdf, cursor, 'Sequence map', () => pdfDrawSequenceMapTable(pdf,
cursor, plan.weeks || []));
  }
  pdfTextSection(pdf, cursor, 'Coverage notes', plan.coverageNotes, true);
  pdfTextSection(pdf, cursor, 'CycleAssess hook',
plan.cycleAssessHooks?.coverageNotes);
  pdfTextSection(pdf, cursor, 'Notes', plan.notes);
}
function pdfAddFooters(pdf){
  const pages = pdf.internal.getNumberOfPages();
  for(let i = 1; i <= pages; i++){
    pdf.setPage(i);
    pdf.setDrawColor(...PDF_TEXT_EXPORT.colours.line);
    pdf.line(PDF_TEXT_EXPORT.margin, 287, PDF_TEXT_EXPORT.w -
PDF_TEXT_EXPORT.margin, 287);
    pdfSetFont(pdf, 8.5, 'normal', PDF_TEXT_EXPORT.colours.muted);
    pdf.text(`CyclePlanner · Page ${i} of ${pages}`, PDF_TEXT_EXPORT.margin, 292);
  }
}
async function exportPdf(){
  if(!state.plan) return setStatus('Create or load a plan
first.', 'warn', 'editStatusBox');
  if(!showPlanQualityWarnings('editStatusBox')) return;
  const jsPdfCtor = window.jspdf && window.jspdf.jsPDF;
  if(!jsPdfCtor){
    setStatus('PDF export library is unavailable here. Opening print instead.',
'warn', 'editStatusBox');
    window.print();
    return;
  }
  setStatus('Building editable text PDF...', 'warn', 'editStatusBox');
  setBusy(true, 'Exporting proper PDF...', 'Writing real PDF text instead of taking
a screenshot. Fancy, but actually useful.');
```

```

  try{
    const pdf = new jsPdfCtor({unit:'mm', format:'a4', orientation:'portrait',
compress:true});

```

```

    const cursor = {y:PDF_TEXT_EXPORT.margin};
    const plan = normalisePlan(clonePlain(state.plan), state.plan.type ||
state.type);
    if(plan.type === 'unit') pdfUnitPlan(pdf, cursor, plan);
    else if(plan.type === 'sequence') pdfSequencePlan(pdf, cursor, plan);
    else pdfLessonPlan(pdf, cursor, plan);
    pdfAddFooters(pdf);
    pdf.setProperties({
      title: pdfPlain(plan.title || PLAN_LABELS[plan.type] || 'CyclePlanner plan'),
      subject: PLAN_LABELS[plan.type] || 'CyclePlanner plan',
      creator: 'CyclePlanner'
    });
    pdf.save(safeFilename(plan.title || PLAN_LABELS[plan.type]) + '.pdf');
    setStatus('Editable text PDF exported.', 'ok','editStatusBox');
  }catch(err){
    console.error(err);
    setStatus('Text PDF export failed. Opening print as fallback.',
'err','editStatusBox');
    window.print();
  }finally{
    setBusy(false);
  }
}

```

```

async function pasteSourceImage(){
  const zone = qs('sourcePasteZone');
  if(navigator.clipboard && navigator.clipboard.read){
    try{
      const count = await addSourceImagesFromClipboardApi(await
navigator.clipboard.read());
      if(count){ setStatus(`${count} pasted image${count === 1 ? '' : 's'} added.`,
'ok'); return; }
    }catch(err){ console.warn('Clipboard read failed:', err); }
  }
  zone?.focus();
  setStatus('Click the paste box and press Ctrl/Cmd + V to paste an image.',
'warn');
}

```

```

/* 10. Event binding */
function startBlank(){
  state.plan = blankPlan(state.type);
  syncCurrentPlanSetup();
  renderAllAndPersist();
  goTab('edit');
}
function buildScaffold(){
  state.plan = scaffoldPlan(state.type);
  syncCurrentPlanSetup();
}

```

```

    renderAllAndPersist();
    goTab('edit');
}
function setEditorCards(open){
    qsa('#editor details.editor-card').forEach(card => { card.open = !!open; });
}
function handleAction(action){
    const actions = {
        'start-blank':startBlank,
        'build-scaffold':buildScaffold,
        'open-templates':openTemplates,
        'close-templates':closeTemplates,
        'generate':generatePlan,
        'choose-source-pdf':()=>qs('sourcePdfInput')?.click(),
        'choose-source-pptx':()=>qs('sourcePptxInput')?.click(),
        'choose-source-image':()=>qs('sourceImageInput')?.click(),
        'paste-source-image':pasteSourceImage,
        'save-history':()=>saveCurrent('history'),
        'save-template':()=>saveCurrent('template'),
        'expand-editor':()=>setEditorCards(true),
        'collapse-editor':()=>setEditorCards(false),
        'copy-text':copyText,
        'export-pdf':exportPdf,
        'print':()=>window.print(),
        'modify-ai':modifyWithAI,
        'make-formal':()=>transformPlan('formal'),
        'make-quick':()=>transformPlan('quick'),
        'copy-studio-brief':copyStudioBrief,
        'key-questions':()=>lightResource('key_questions'),
        'exit-ticket':()=>lightResource('exit_ticket'),
        'misconceptions':()=>lightResource('misconceptions'),
        'clear-modify':()=>{qs('modifyRequest').value='';},
        'clear-library':clearLibrary,
        'save-settings':saveSettings
    };
    actions[action]?.(());
}
function bindEvents(){
    document.addEventListener('click', event=>{
        const cycleChoice = event.target.closest('[data-cycle-choice]');
        if(cycleChoice){
            applyCyclePreference(cycleChoice.dataset.cycleChoice, qs('yearLevel')?.value
|| '');
            savePrefs();
            qs('cyclePreferenceModal')?.classList.remove('open');
            return;
        }
        const go = event.target.closest('[data-go]');
        if(go){goTab(go.dataset.go);return;}
        const sourceRemove = event.target.closest('[data-source-remove]');

```

```

    if(sourceRemove){removeSourceFile(sourceRemove.dataset.sourceRemove);return;}
    const action = event.target.closest('[data-action]');
    if(action){handleAction(action.dataset.action);return;}
    const load = event.target.closest('[data-load-id]');
    if(load){loadSaved(load.dataset.loadKey, load.dataset.loadId);return;}
    const del = event.target.closest('[data-delete-id]');
    if(del){deleteSaved(del.dataset.deleteKey, del.dataset.deleteId);return;}
    const template = event.target.closest('[data-template-id]');
    if(template){useTemplate(template.dataset.templateId);return;}
    const add = event.target.closest('[data-add-row]');
    if(add){addRow(add.dataset.addRow);return;}
    const rem = event.target.closest('[data-remove-row]');
    if(rem){removeRow(rem.dataset.removeRow, rem.dataset.index);return;}
  });
  document.addEventListener('input', event=>{
    if(event.target.matches('#durationCustom')){ handleDurationChange(); return; }
    if(event.target.matches('#unitLessonCustom')){ handleCountChange('unitLesson');
return; }
    if(event.target.matches('#sequenceWeekCustom')){
handleCountChange('sequenceWeek'); return; }

if(event.target.matches('#yearLevel,#briefInput,#extraContextInput,#sourceDetailsIn
put,#apiKey,#modelName,#defaultContext')){ savePrefsSoon(); renderCreateSummary();
}
    if(event.target.dataset.path && state.plan){
      setNested(state.plan, event.target.dataset.path, event.target.value);
      schedulePreviewUpdate();
    }
    if(event.target.dataset.arrayPath && state.plan){
      setNested(state.plan, event.target.dataset.arrayPath,
splitLines(event.target.value));
      schedulePreviewUpdate();
    }
  });
  document.addEventListener('change', event=>{
    if(event.target.id === 'durationPreset'){ handleDurationChange(); return; }
    if(event.target.id === 'unitLessonPreset'){ handleCountChange('unitLesson');
return; }
    if(event.target.id === 'sequenceWeekPreset'){
handleCountChange('sequenceWeek'); return; }
    if(event.target.id === 'sourcePdfInput'){
      addSourceFiles(event.target.files).then(count=>setStatus(count ? `${count}
PDF source added.` : 'No PDF source added.', count ? 'ok' : 'warn'));
      event.target.value = '';
      return;
    }
    if(event.target.id === 'sourcePptxInput'){
      addSourceFiles(event.target.files)
        .then(count=>setStatus(count ? `${count} PPTX source added.` : 'No PPTX
source added.', count ? 'ok' : 'warn'))

```

```

        .catch(err=>setStatus(err.message || 'Could not add PPTX source.', 'err'));
        event.target.value = '';
        return;
    }
    if(event.target.id === 'sourceImageInput'){
        addSourceFiles(event.target.files).then(count=>setStatus(count ? `${count}
image source${count === 1 ? '' : 's'} added.` : 'No image source added.', count ?
'ok' : 'warn'));
        event.target.value = '';
        return;
    }
    if(event.target.id === 'planningTypeSelect')
selectPlanningType(event.target.value);
    if(event.target.id === 'preferredCycle'){
        applyCyclePreference(event.target.value, qs('yearLevel')?.value || '');
        savePrefs();
        return;
    }
    if(event.target.id === 'yearLevel'){
        savePrefs();
        renderCreateSummary();
        return;
    }
    if(event.target.id === 'domainSelect'){
        state.selectedOutcomes = [];
        renderCurriculumMode();
        savePrefs();
    }
    if(event.target.matches('[data-outcome]')){
        const value = event.target.dataset.outcome;
        if(event.target.checked) state.selectedOutcomes =
unique([...state.selectedOutcomes,value]);
        else state.selectedOutcomes = state.selectedOutcomes.filter(outcome=>outcome
!== value);
        savePrefs();
    }
});
const sourcePasteZone = qs('sourcePasteZone');
if(sourcePasteZone){

sourcePasteZone.addEventListener('focus',()=>sourcePasteZone.classList.add('active'
));

sourcePasteZone.addEventListener('blur',()=>sourcePasteZone.classList.remove('activ
e'));
    sourcePasteZone.addEventListener('click',()=>sourcePasteZone.focus());
    sourcePasteZone.addEventListener('paste', async event=>{
        const count = await
addSourceImagesFromClipboardItems(event.clipboardData?.items || []);
        if(count){

```

```

        event.preventDefault();
        setStatus(`${count} pasted image${count === 1 ? '' : 's'} added.`, 'ok');
    }else{
        setStatus('No image found in the clipboard.', 'warn');
    }
});
}
qsa('[data-curr]').forEach(btn=>btn.addEventListener('click',()=>{
    state.curriculumMode = btn.dataset.curr;
    renderCurriculumMode();
    savePrefs();
}));
}
function init(){
    const shouldAskCycle = !hasStoredCyclePreference();
    loadPrefs();
    const savedDomain = getPrefs().domain || 'English Literacy';
    qs('domainSelect').value = DOMAIN_LIBRARY[savedDomain] &&
domainHasOutcomes(savedDomain) ? savedDomain : (availableDomainNames()[0] ||
DOMAIN_NAMES[0] || '');
    selectPlanningType(state.type);
    renderCurriculumMode();
    renderCreateSummary();
    renderSourceFiles();
    const current = readJSON(STORAGE.current, null);
    if(current){
        state.type = current.type || state.type;
        state.plan = normalisePlan(current, state.type);
        selectPlanningType(state.type);
    }
    renderAll();
    bindEvents();
    if(shouldAskCycle) qs('cyclePreferenceModal')?.classList.add('open');
}
window.addEventListener('DOMContentLoaded', function(){
    if(window.SylCycle && SylCycle.onReady) SylCycle.onReady(init);
    else init();
});
</script>
</body>
</html>

```